GRAU EN ENGINYERIA INFORMÀTICA

102753 Visualització Gràfica Interactiva



3.3+.

2. Shaders

Enric Martí Gòdia

Departament de Ciències de la Computació

Continguts

1. Introducció
2. Graphics Pipeline
3. Llenguatges per a shaders
4. Llenguatge GLSL
5. Exemples de Shaders
6. Geometry Shader (GS)

1. Introducció

Shader: Procediment que permet al programador especificar un renderitzat d'un vèrtex o un píxel (Renderman, Pixar, 1989).

1999: Apareix GeForce 256 de nVidia que inclou GPU's (*Graphics Processing Unit*) programables.

Shaders: Programes que transformen, un vèrtex, una geometria o un píxel (fragment). Substitueixen el *Fixed Funcion Pipeline* (FFP) d'OpenGL v.1.1, permetent al programador visualitzar qualsevol efecte.

1. Introducció

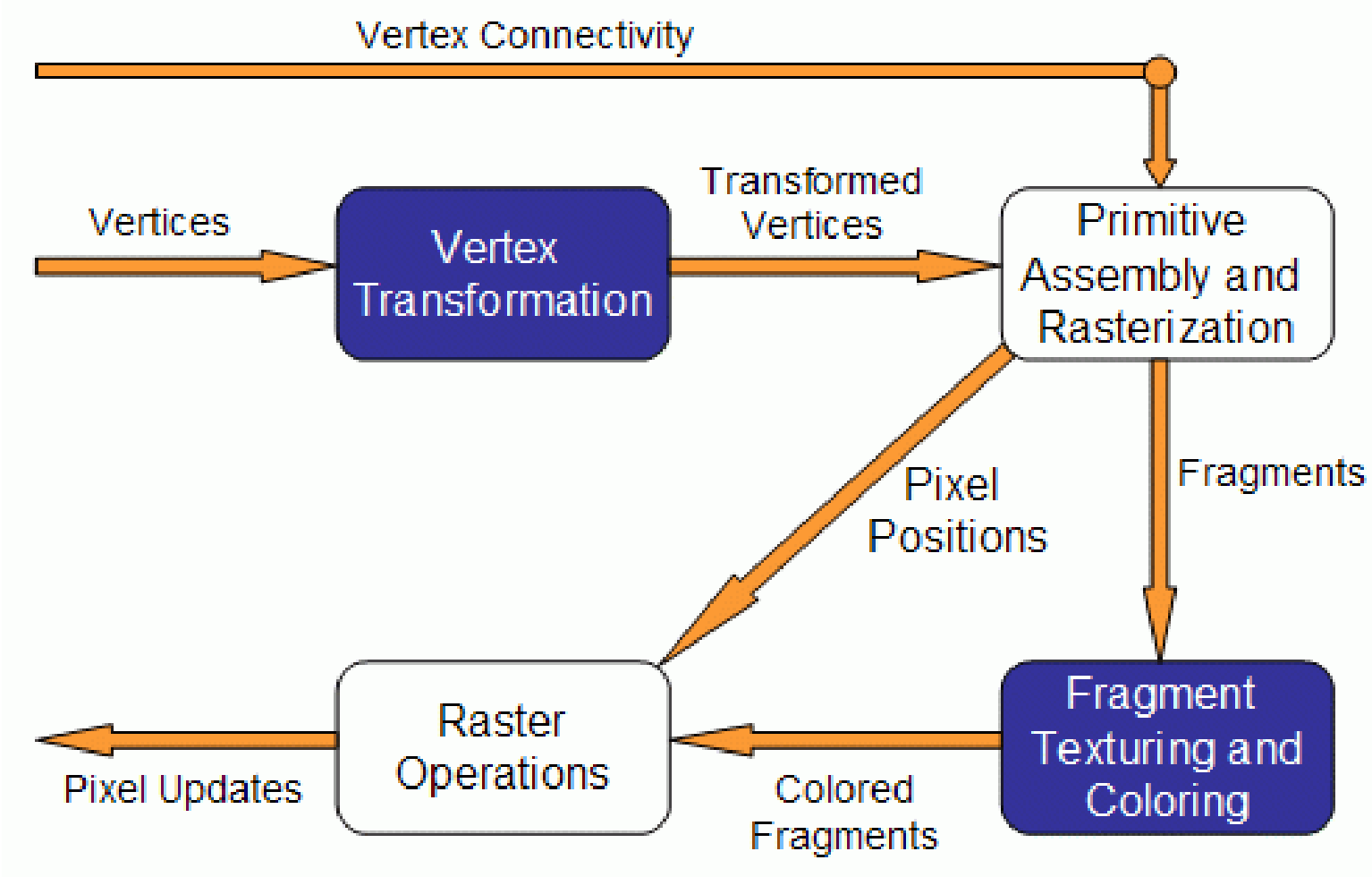
Shaders: S'utilitzen per a efectes de:

- Iluminació
- Ombres
- Animacions
- Efectes de partícules, etc.

Tres shaders, entre d'altres:

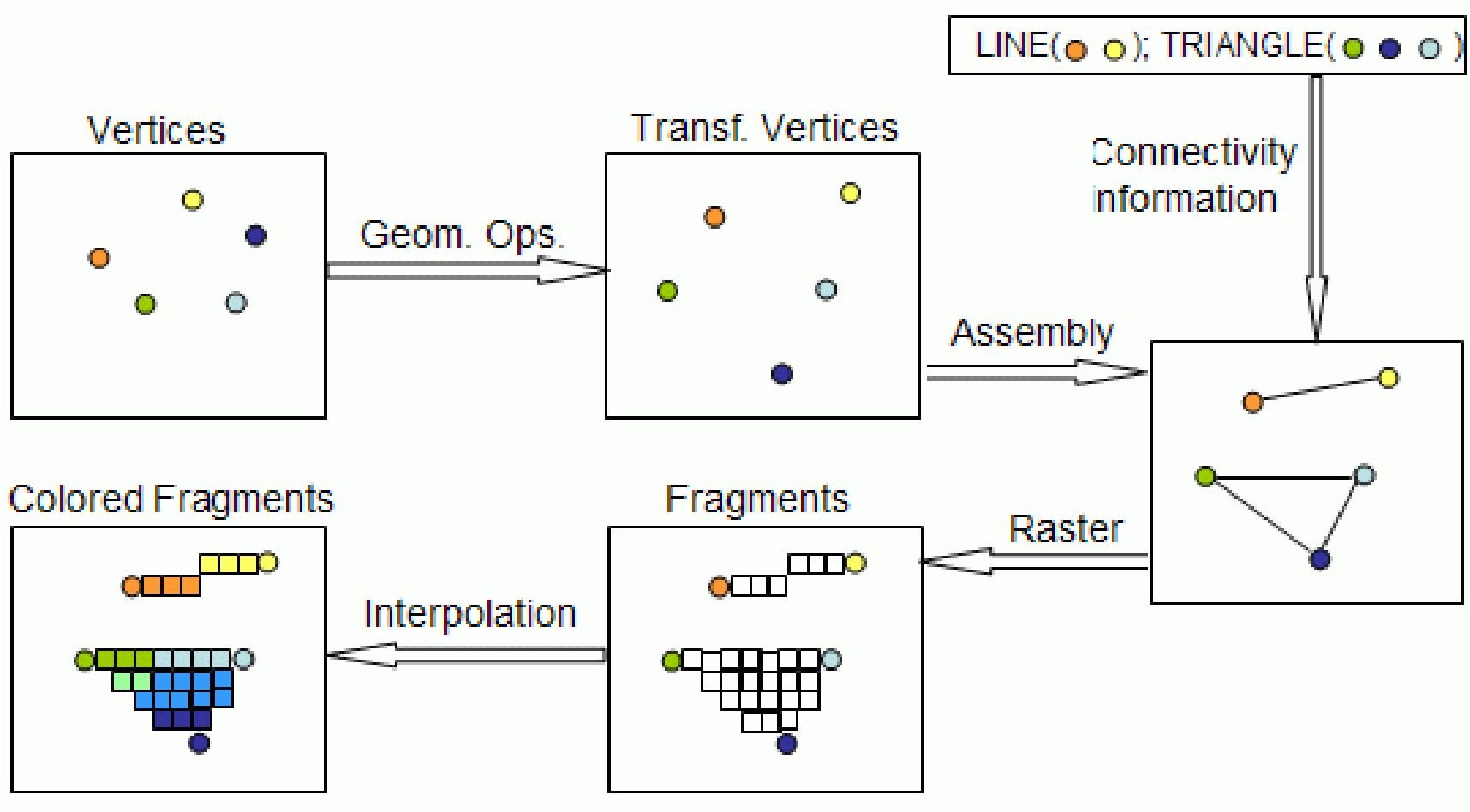
- *Vertex Shader* (VS)
- *Fragment Shader* o *Pixel Shader* (PS)
- *Geometry Shader (GS)*

Graphics Pipeline



2. Graphics Pipeline

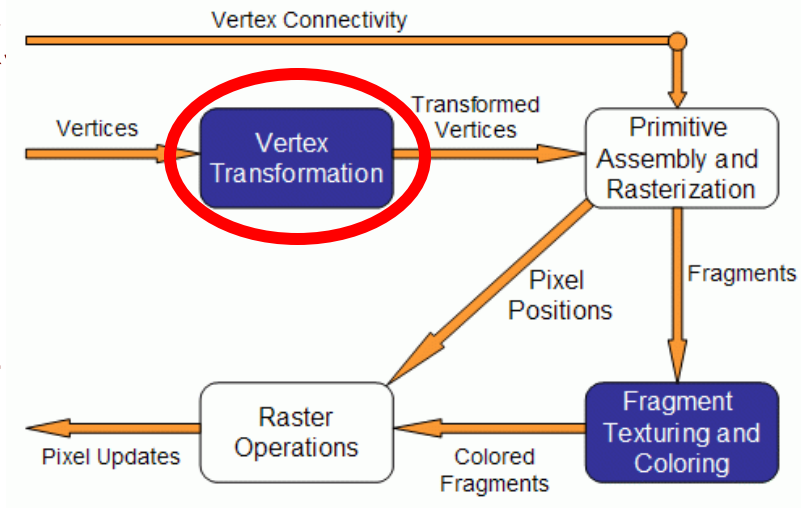
Visual Pipeline



2. Graphics Pipeline

Vertex Shader (VS): Programa que manipula vèrtexs en 3D. Càlculs sobre els vèrtex:

- Posició final
- Color dels polígons
- Interpretar coordenades textura



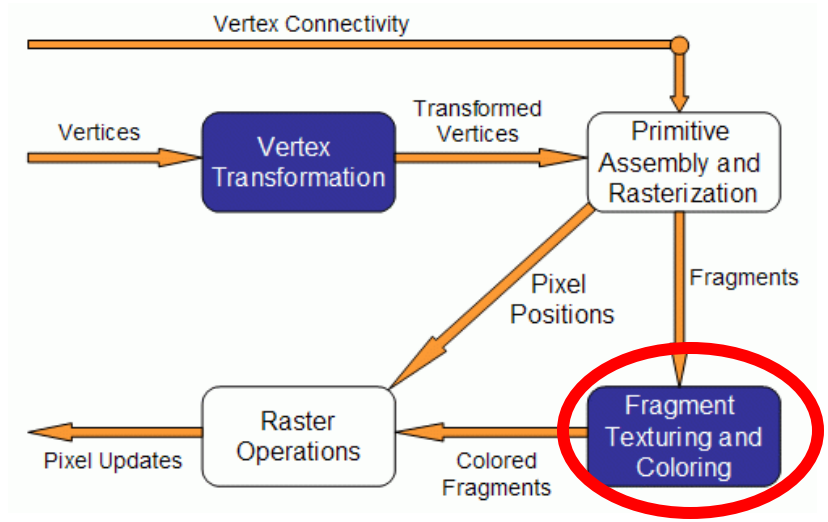
VS opera sobre cada vèrtex, no en crea de nous ni en destrueix, sols els modifica.

2. Graphics Pipeline

Fragment Shader (FS): Programa que manipula píxels o fragments de pantalla. Necessita les dades calculades pel VS.

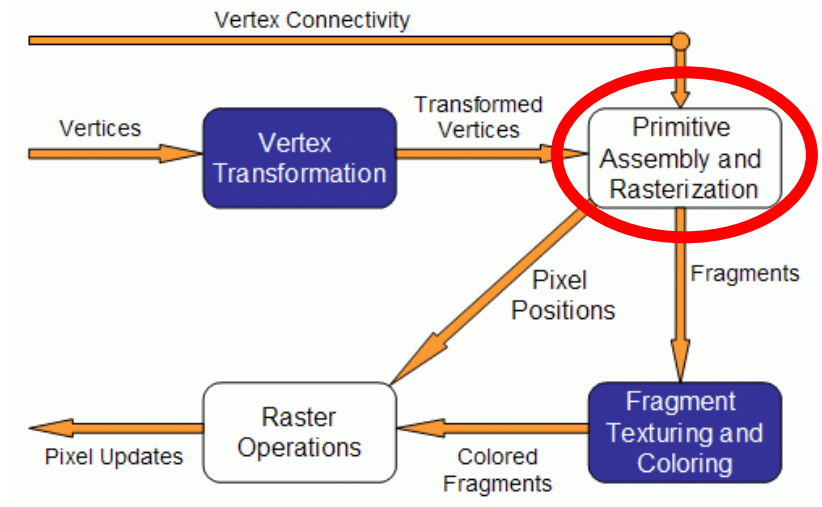
Càlcul del color final dels fragments en funció de:

- Orientació del triangle
- Orientació vector de llum
- Orientació del vector de vista
- Textura



2. Graphics Pipeline

Geometry Shader (GS): Programa que controla la formació o eliminació de primitives abans del *Primitive Assembly.*

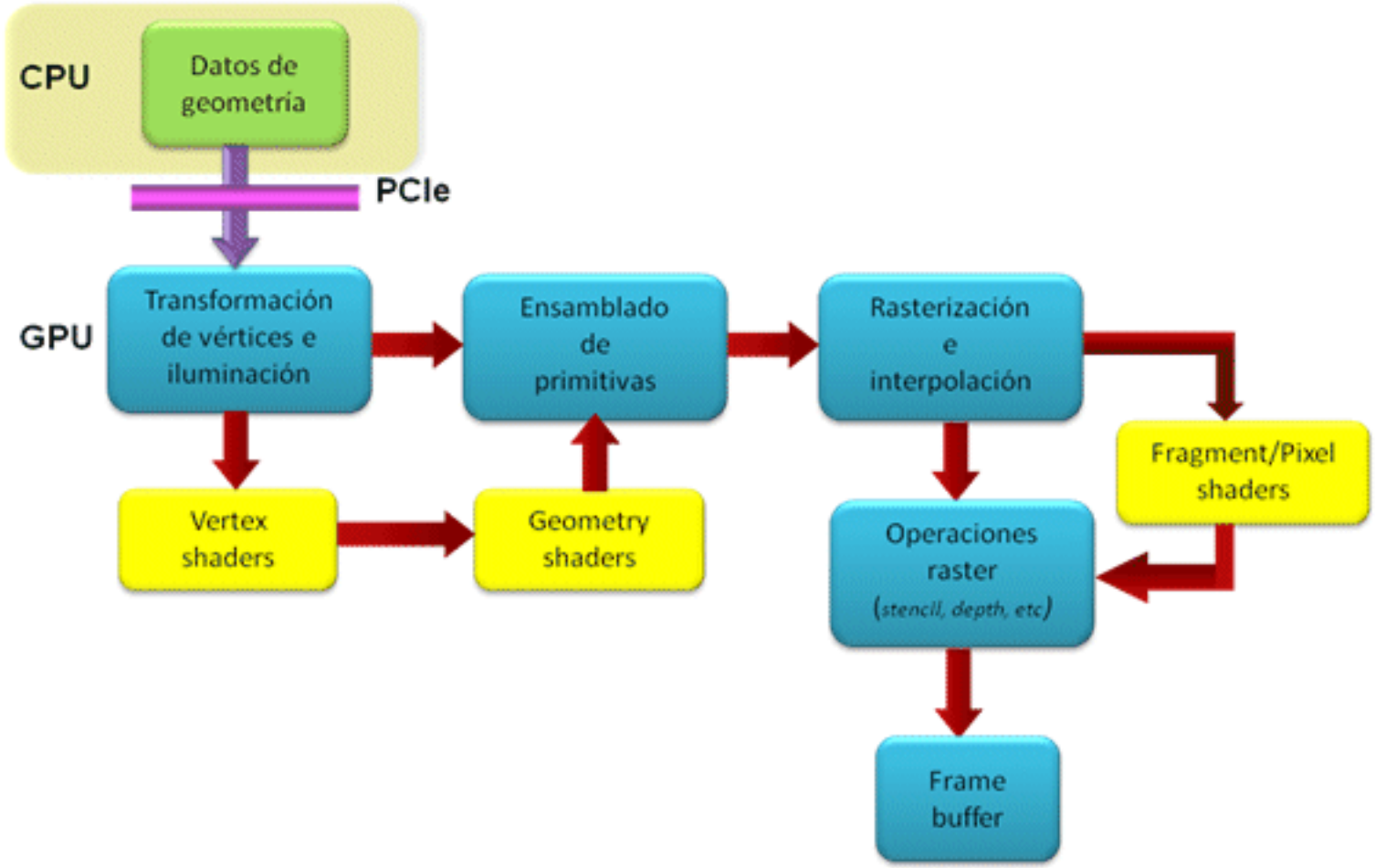


Generar geometria dinàmicament.

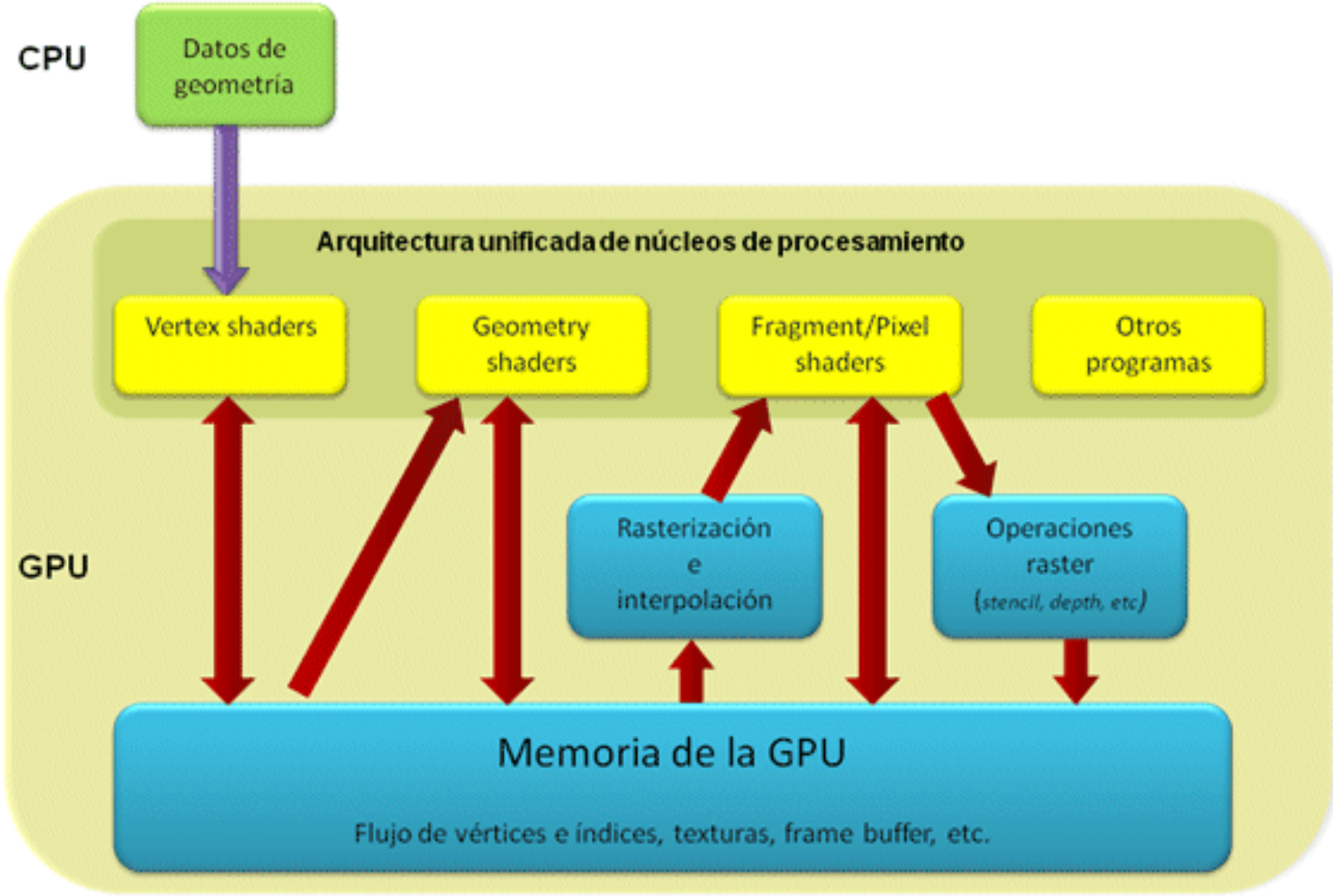
Un GS pot fer:

- Crear noves primitives
- Extensió `GL_EXT_geometry_shader4`.

2. Graphics Pipeline



2. Graphics Pipeline



3. Llenguatges per a shaders

Tres tipus de llenguatges segons la llibreria gràfica que les sustenta:

- **GLSL (*OpenGL Shading Language*) - OpenGL**
- **HLSL (*High level Shading language*) - Direct 3D**
- **Nvidia Cg (*C for Graphics*) – OpenGL i Direct 3D**

Tots tres tenen sintaxi molt semblant entre ells basada en llenguatge C.

3. Llenguatges per a shaders

GLSL (*OpenGL Shading Language*): Creat per la *OpenGL Architecture Review Board* (ARB).

Avantatges:

- Compatibilitat multi-plataforma en diferents Sistemes Operatius com GNU/Linux, Mac OS X i Windows.
- Capacitat d'escriure *programs* de shader que poden ser utilitzats en diferents models de hardware gràfic i de diferents fabricants (NVIDIA, ATI, Intel, etc.) que suportin GLSL.
- Cada fabricant de hardware gràfic inclou el seu compilador GLSL en el driver, el que li permet implementar codi optimitzat sobre l'arquitectura del seu hardware gràfic.

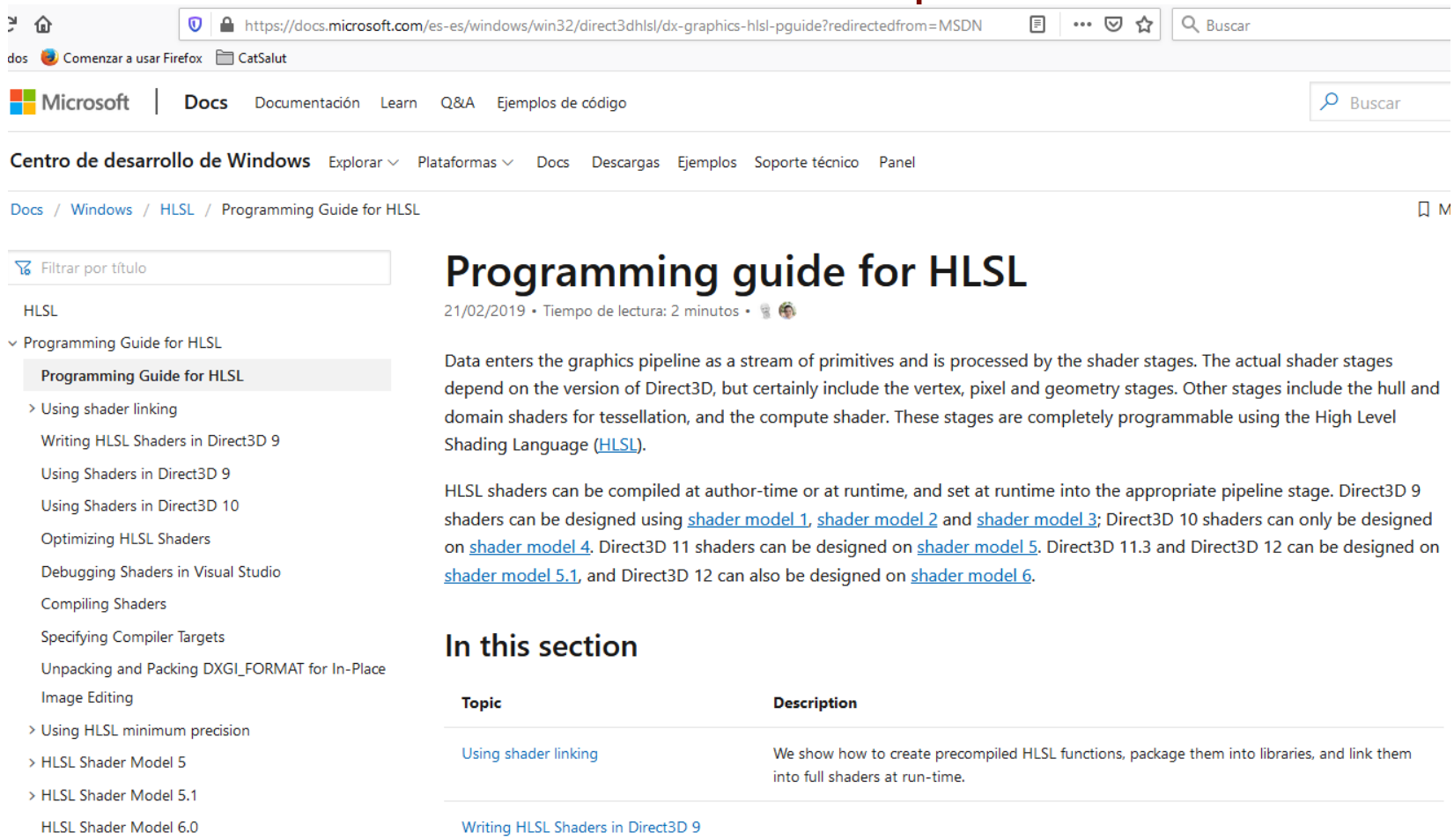
3. Llenguatges per a shaders

GLSL (*OpenGL Shading Language*)

GLSL Version	OpenGL Version	Date	Shader Preprocessor
1.10.59	2.0	30 April 2004	#version 110
1.20.8	2.1	07 September 2006	#version 120
1.30.10	3.0	22 November 2009	#version 130
1.40.08	3.1	22 November 2009	#version 140
1.50.11	3.2	04 December 2009	#version 150
3.30.6	3.3	11 March 2010	#version 330
4.00.9	4.0	24 July 2010	#version 400
4.10.6	4.1	24 July 2010	#version 410
4.20.11	4.2	12 December 2011	#version 420
4.30.8	4.3	7 February 2013	#version 430
4.40.9	4.4	16 June 2014	#version 440
4.50.7	4.5	09 May 2017	#version 450
4.60.5	4.6	14 June 2018	#version 460

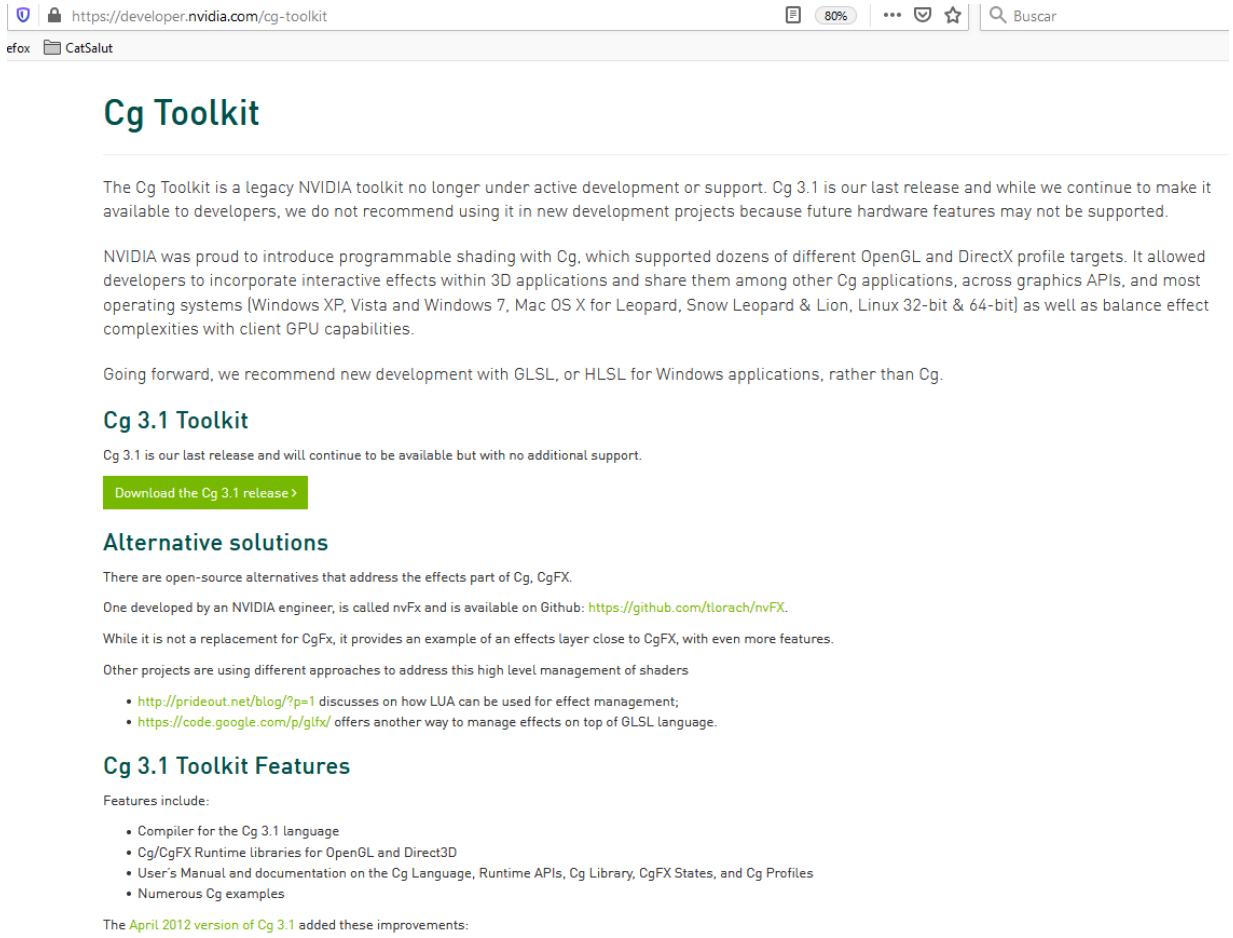
3. Llenguatges per a shaders

HLSL (*High Level Shading Language*): Creat per Microsoft amb colaboració amb NVidia per a llibreria Direct 3D.



3. Llenguatges per a shaders

Cg (*C for Graphics*): Llenguatge creat per Nvidia, en un intent de ser multiplataforma entre OpenGL i Direct 3D.



4. GLSL

Llenguatge amb sintaxi C.

Codi per a càlculs de vectors i matrius.

Estructura típica codi font. Blocs de codi:

```
// Bloc 1: Capçalera. Versió compilador, definició constants i structs
#version version_number
#define name const;
struct {}

// Bloc 2 Definició variables in, out, uniform
in type in_nom_variable1;
in type in_nom_variable_2;
out type out_nom_variable;
uniform type nom_uniform;

// Bloc 3: Codi principal
void main()
{
// Càlculs a partir de les variables d'entrada (in i uniforms)
<càlculs> = funcio (in_nom_variable1, in_nom_variable2,nom_uniform);
...
// Definició variables sortida (out) a partir dels càlculs realitzats
out_nom_variable = <càlculs>;
}
```

4. GLSL

En Vertex shader, una variables d'entrada (in) provinent de l'aplicació s'anomena *vertex attribute*.

Cada implementació OpenGL en hardware gràfic defineix un màxim nombre de *vertex attributes*, com a mínim 16 variables de 4 components `(x,y,z,w)`.

En l'aplicació es pot consultar aquest nombre màxim amb la comanda:

```
GLint nrAttributes;
glGetIntegerv(GL_MAX_VERTEX_ATTRIBS, &nrAttributes);
fprintf(err,"Màxim nombre de vertex attributes:, %i \n", nrAttributes);
```

4. GLSL

Continguts:

1. Tipus
2. Variables
3. Sentències i funcions
4. Exemple VS - FS
5. Compilació de shaders
6. Shader Debugging

4.1. Tipus GLSL

GLSL suporta tipus molts semblants a C i C++:

1. Tipus bàsics
2. Arrays
3. Structs

4.1.1. Tipus bàsics GLSL

GLSL suporta tipus simples de C: `bool`, `int`, `uint`, `float`, `double`, etc.)

Tipus `double` incorporat a partir GL4.0 o amb l'extensió `ARB_gpu_shader_fp64`

GLSL soporta tres tipus bàsics més complexes:

a) Vectors

b) Matrius

c) Tipus opacs

4.1.1. Tipus bàsics GLSL: Vectors

Contenidor de 1, 2, 3 o 4 components:

- `vecn`: vector n `float`
- `bvecn`: vector n `bool`
- `ivecn`: vector n `int`
- `uvecn`: vector n `unsigned int`
- `dvecn`: vector n `double`

4.1.1. Tipus bàsics GLSL: Vectors

Accés a components del vector a través **swizzling**:

- `.s .r .x` per a la primera component
- `.t .g .y` per a la segona component
- `.p .b .z` per a la tercera component
- `.q .a .w` per a la quarta component

Per a vectors:

- `(.x,.y,.z,.w)` per a vèrtex i vector normal
- `(.r,.g,.b,.a)` per a color
- `(.s,.t,.p,.q)` per a coordenada textura

Swizzling dona molta flexibilitat en selecció de components d'un vector:

```
vec2 unVec = vec2(0.0,1.0);        // unVec.zw no existeixen. ERROR
vec4 diffVec = unVec.xyxx;
vec3 altraVec = diffVec.zyw;
vec4 altra2Vec = unVec.xxxx + altraVec.yxzy;
```

Podem crear nous vectors a partir de components d'altres:

```
vec2 unVector = vec2(0.5,0.5);
vec4 resultat = vec4(unVector,0.0,1.0);
vec4 resultat2 = vec4(resultat.xyz,1.0);
```

4.1.1. Tipus bàsics GLSL: Matrius

Contenidor de 2, 3 o 4 components:

- `matnxm`: Matriu de n columnes i m files de tipus `float`
- `dmatnxm`: Matriu de n columnes i m files de tipus `double` (a partir de GL 4.0).
- `matn`: Matriu de n files i n columnes `float`. Equivalent a `dmatnxn`
- `dmatn`: Matriu de n files i n columnes `double`. Equivalent a `dmatnxn` (a partir de GL 4.0).

4.1.1. Tipus bàsics GLSL: Matrius

També accés a columnes de la matriu a través **swizzling**:

- `.x` per a la primera component de la columna
- `.y` per a la segona component de la columna
- `.z` per a la tercera component de la columna
- `.w` per a la quarta component de la columna

Swizzling dona molta flexibilitat en selecció de components de la columna:

```
mat3 Matrix;            // Matrix[0].w no existeix. ERROR
Matrix[1] = vec3(1.0, 2.0, 3.0); // Segona columna matriu
Matrix[2][0] = 8.0;     // Primera component tercera columna matriu
Matrix[3].yzx = vec3(4.0, -5.0, 1.0);
```

4.1.1. Tipus bàsics GLSL: Matrius

Constructors:

Amb un sol valor, s'inicialitza la diagonal de la matriu:

```
mat3 identMatrix = mat3(1.0); // Matriu identitat.
mat3 diagMatrix = mat3(5.0);  // Matriu amb valor 5.0 a la diagonal.
mat4 otherMatrix = mat4(diagMatrix); // Diagonal amb 3 valors 5.0 i
                                     //    el darrer 1.0.
```

Amb diferents valors, la matriu s'emplena per columnes:

```
mat2( float, float,   // Primera columna
      float, float);  // Segona columna
mat4( vec4,           // Primera columna
      vec4,           // Segona columna
      vec4,           // Tercera columna
      vec4);          // Quarta columna
mat3( vec2, float,    // Primera columna
      vec2, float,    // Segona columna
      vec2, float);   // Tercera columna
```

4.1.1. Tipus bàsics GLSL: Tipus Opacs

Representen un objecte extern. No tenen valors, referencien una dada. Únicament s'utilitzen com a paràmetres a funcions.

Només es poden declarar de dues formes: com variables `uniform` o com a paràmetre d'una funció.

GLSL soporta tres tipus opacs:

a) Sampler (textura)

b) Image (imatge)

c) *Atomic counters* (adreça a *Buffer Objects*)

4.1.1. Tipus bàsics GLSL: Tipus Opacs. Sampler

Representen l'objecte textura.

Tres tipus de textura:

- Imatge (`isampler*, uisampler*, sampler*`)
- Depth-component (`sampler*`)
- Stencil-component (`usampler*`)

Imatge (*g* pot ser `i`, `ui` o en blanc):

GLSL sampler	OpenGL texture enum	Texture type
g`sampler2D`	`GL_TEXTURE_2D`	2D texture
g`sampler3D`	`GL_TEXTURE_3D`	3D texture
g`samplerCube`	`GL_TEXTURE_CUBE_MAP`	Cubemap texture

4.1.1. Tipus bàsics GLSL: Tipus Opacs. Sampler

Depth-component o Stencil-component: S'utilitzen per a comparacions de profunditat, per ombres.

component (*g* pot ser u o en blanc):

GLSL sampler	OpenGL texture enum
`gsampler1DShadow`	`GL_TEXTURE_1D`
`gsampler2DShadow`	`GL_TEXTURE_2D`
`gsamplerCubeShadow`	`GL_TEXTURE_CUBE_MAP`

Definició de samplers: Com variables *uniform* o com a paràmetre d'una funció:

```
uniform sampler2D texture1;

void Function(in sampler2D myTexture);
```

4.1.1. Tipus bàsics GLSL: Tipus Opacs. Imatge

Imatge de lectura/escriptura per a fer efectes de barreja (*blending*).

Tres tipus amb prefixes:

- `int (iimage*)`
- `Unsigned int (uimage*)`
- `float (image*)`

Imatge (*g* pot ser `i`, `ui` o en blanc):

Image Type	Texture Type
*g*image2D	GL_TEXTURE_2D
*g*image3D	GL_TEXTURE_3D
*g*imageCube	GL_TEXTURE_CUBE_MAP

4.1.2. Arrays GLSL

Semblants a arrays C/C++, però amb algunes limitacions:

- Arrays han de tenir un tamany, inicialitzat amb una expressió constant, no variable.
- Poden ser definits d'entrada per a qualsevol shader (vertex, geometry o fragment), però no poden ser definits com a sortida del fragment shader.
- Longitut d'un array es pot obtenir amb la funció `.length()`

```
uniform sampler images[10];
uniform int imageCount;
void main()
{
vec4 accum = vec4(0.0);
for (int i = 0; i < imageCount; i++) {    accum += texture(images[i], ...);
                                         }
images.length();      // Retorna 10.
}
```

4.1.2. Arrays GLSL

Arrays d'arrays: Es poden declarar arrays multidimensionals a partir de GL4.3+.

Només el primer índex pot ser sense tamany.

```
uniform vec3 multidim[5][2];     // Correcte.
uniform vec3[5][2] multidimA;    // Correcte.
vec3[][2] multidimB;             // Correcte.
vec3[5][] multidimC;             // No correcte.

multidim.length();       // Retorna 5.
multidim[0].length();    // Retorna 2.
```

4.1.2. Arrays GLSL

Constructors: S'emplenen utilitzant la mateixa sintaxi de l'array.

```
const float array[3] = float[3](2.5, 7.0, 1.5);
```

4.1.3. Structs GLSL

Semblants a structs C/C++, però amb algunes limitacions:

- No es soporta l'estructura `typedef struct +{ }`
- Tot `struct` ha de tenir un membre com a mínim
- Un `struct` no pot ser definit dins un altra `struct`, però un `struct` pot utilitzar un altra prèviament definit

```
struct Color
{ vec3 diffuse;
  vec3 specular;
}
struct Light
{ vec3 eyePosOrDir;
  bool isDirectional;
  Color intensity;
  float attenuation;
} variableName;}
variableName.intensity.diffuse = vec3(1.0,0.7,0.2);
```

4.1.3. Structs GLSL

Constructors: S'emplenen en el mateix ordre en que són definits.

```
struct Data
{ float first;
  vec2 second;
};

Data dataValue = Data(1.4, vec2(16.0, 22.5));
```

4.2. Variables GLSL

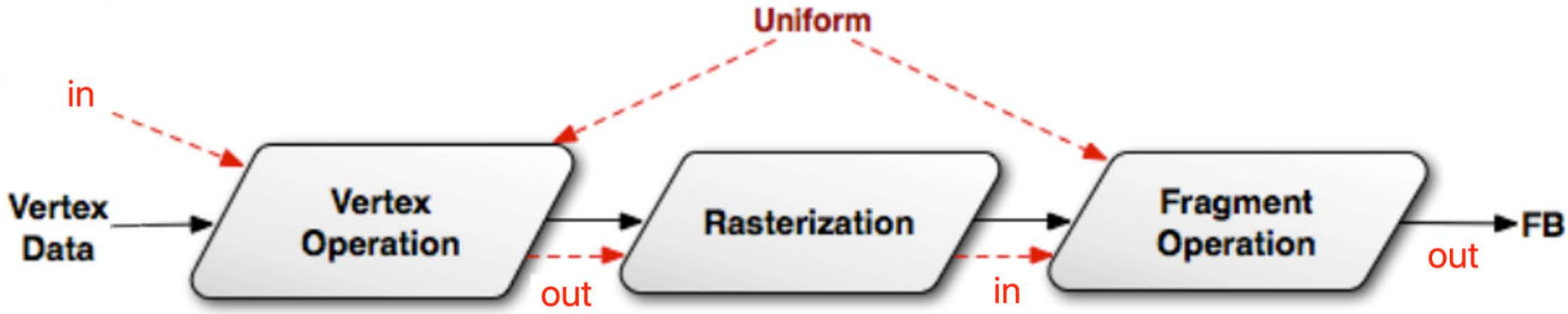
Cada programa shader defineix variables simples de C: `bool`, `int`, `uint`, `float`, `double`, etc.).

Tipus `double` incorporat a partir GL4.0 o amb l'extensió `ARB_gpu_shader_fp64`.

4.2. Variables GLSL

Els shaders treballen amb dos tipus de variables que connecten continguts amb l'aplicació:

- **in-out:** Variables d'entrada i sortida al shader.
- **uniforms:** Variables d'entrada a tots els shaders. Valors fixes provinents de l'aplicació.



4.2.1. Variables in-out GLSL

Variables in: Variables d'entrada al shader, provinents de l'aplicació (*vertex shader*) o calculades d'anteriors shaders (*geometry* o *fragment shader*).

Variables out: Variables de sortida calculades dins del shader que van a un altra shader (*vertex* o *geometry shader*) o a pantalla (*fragment shader*).

Es declaren explícitament al shader en el bloc de definicions:

```
#version 330 core
in a;
out b;
b=a; // Si variable sortida equival a entrada, cal assignar.
```

4.2.1. Variables in-out GLSL

Vertex i Fragment shader difereixen en la definició i manipulació de variables:

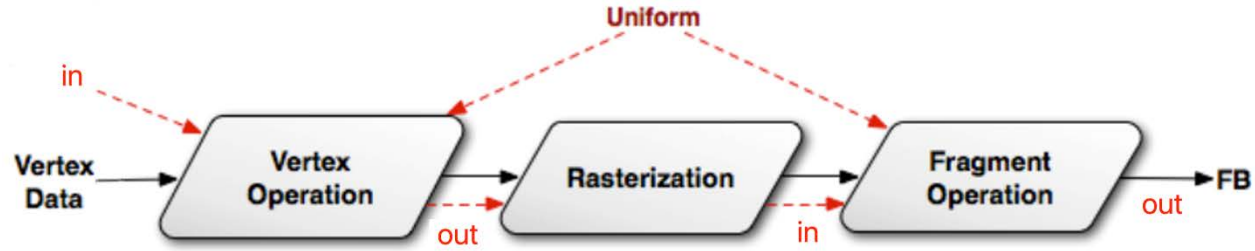
Vertex shader: Les variables d'entrada corresponen a buffers VBO, VAO o EBO (com a mínim la posició dels vèrtexs del model 3D). Variables especificades mitjançant adreça (comanda `layout`).

Fragment shader: Necessari definir una variable `vec4` per a definir color final del fragment. Si no s'especifica una variable de sortida → Fons negre

4.2.1. Variables in-out GLSL

Podem relacionar variables entre shaders. L'entrada i la sortida han de coincidir en nom i tipus.

```
#version 330 core // Vertex shader
layout (location=0) in vec3 position;
out vec3 vertexColor;
void main()
{ gl_position = vec4(position,1.0);
  vertexColor = vec4(0.5f,0.0f,0.0f);
}
#version 330 core // Fragment shader
in vec3 vertexColor;
out vec4 FragColor;
void main()
{ FragColor = vertexColor;
}
```



Valor d'entrada al fragment interpolat a partir de valors de sortida calculats en el vertex shader.

4.2.2 Variables uniform GLSL

Variables accessibles a tots els shaders, no modificables. El seu valor prové de l'aplicació.

```
#version 330 core // Fragment shader
uniform vec4 ourColor;
out vec4 FragColor;
void main()
{ FragColor = ourColor;
}
```

Si la variable `uniform` no es referencia en un shader no cal declarar-la.

Poden ser qualsevol tipus GLSL: tipus bàsics (`int`,`float`), vectors (`vec`) i matrius (`mat`) o arrays i struct.

4.2.2 Variables uniform GLSL

Pas de valors de l'aplicació a variables uniform.

Passos:

1. Shader compilat, linkat i activat amb un identificador (`uint shader_program`)
2. Obtenir l'adreça de la variable uniform dins el shader → `glGetUniformLocation()`
3. Assignar una variable de l'aplicació a l'adreça de la variable uniform:
   - Si variables de tipus bàsics → `glUniform*()`
   - Si variables tipus matriu → `glUniformMatrix()`

4.2.2 Variables uniform GLSL

```
GLint glGetUniformLocation(GLuint program, const GLchar *name );
```

Acció: Retorna un `GLint` que representa l'adreça de la variable uniform referenciada a `name` del shader identificat amb `program`.

Paràmetres:

`program`: Identificador del shader program que conté la variable uniform.

`name`: Nom de la variable uniform.

4.2.2 Variables uniform GLSL

```
void glUniform1{f,i,ui}(GLint location,{GLfloat,GLint,GLuint} v0);
void glUniform2{f,i,ui}(GLint location,{GLfloat,GLint,GLuint} v0,{GLfloat,GLint,GLuint}
v1);
void glUniform3{f,i,ui}(GLint location,{GLfloat,Glint,GLuint} v0,{GLfloat,GLint,GLuint}
v1,      {GLfloat,GLint,GLuint} v2);
void glUniform4{f,i,ui}(GLint location,{GLfloat,GLint,GLuint} v0,
{GLfloat,GLint,GLuint}      v1, {GLfloat,GLint,GLuint} v2, {GLfloat,GLint,GLuint} v3);
void glUniform{1,2,3,4}{i,f,ui}v(GLint location, GLsizei count, const
{GLfloat,GLint,GLuint} *v);
```

Acció: Assigna una variable `uniform` de GLSL a una variable de l'aplicació.

Paràmetres:

`location`: Adreça de la variable uniform del shader actiu.

`v0,v1,v2,v3`: Valors constants o variables de l'aplicació a assignar a la variable uniform.

`count`: Nombre d'elements assignar (comanda `glUniform*v()`).

`v`: Apuntador de la variable de l'aplicació amb un array de valors (comanda `glUniform*v()`).

4.2.2 Variables uniform GLSL

```
void glUniformMatrix{2,3,4}fv(GLint location, GLsizei count, GLboolean
transpose, const GLfloat *v);

void glUniformMatrix{2,3,4}x{2,3,4}fv}(GLint location, GLsizei count,
GLboolean transpose, const GLfloat *v);
```

Acció: Assigna a la variable `matn` de GLSL el valor d'una variable mtriu de l'aplicació.

Paràmetres:

`location`: Adreça de la variable uniform del shader actiu.

`count`: Nombre d'elements assignar.

`transpose`: Si trasposar la matriu quan s'assignin els valors a la variable `uniform` (valor `true`) o no.

`v`: Apuntador a la matriu de valors.

4.2.2 Variables uniform GLSL

Exemple. Vertex o Fragment shader amb variables:

```
uniform float specIntensity;
uniform vec4 specColor;
uniform float t[2];
uniform vec4 color[3];
```

Codi de l'aplicació per al pas de valors a les variables `uniform`:

```
GLint loc1,loc2,loc3,loc4;
float specIntensitat = 0.98;
float spCol[4] = {0.8,0.8,0.8,1.0};
float threshold[2] = {0.5,0.25};
float colors[12] = {0.4,0.4,0.8,1.0, 0.2,0.2,0.4,1.0, 0.1,0.1,0.1,1.0};

glUseProgram(shaderProgram);          // Activar Programa Shader
loc1 = glGetUniformLocation(shaderProgram,"specIntensity");
glUniform1f(loc1,specIntensitat);
loc2 = glGetUniformLocation(shaderProgram,"specColor");
glUniform4fv(loc2,1,spCol); | glUniform4f(loc2,spCol[0],spCol[1],spCol[2],spCol[3]);
loc3 = glGetUniformLocation(shaderProgram,"t");
glUniform1fv(loc3,2,threshold);
loc4 = glGetUniformLocation(shaderProgram,"color");
glUniform4fv(loc4,3,colors);
```

4.2.2. Variables uniform GLSL

Exemple. Shader amb variables:

```
uniform float specIntensity;
uniform vec4 specColor;
uniform float t[2];
uniform vec4 color[3];
```

Codi de l'aplicació per al pas de valors a les variables `uniform`:

```
glUseProgram(shaderProgram);          // Activar Programa Shader
float spIntensitat = 0.98;
float spCol[4] = {0.8,0.8,0.8,1.0};
float threshold[2] = {0.5,0.25};
float colors[12] = {0.4,0.4,0.8,1.0, 0.2,0.2,0.4,1.0, 0.1,0.1,0.1,1.0};

glUniform1f(glGetUniformLocation(shaderProgram,"specIntensity"),specIntensitat);
glUniform4fv(glGetUniformLocation(shaderProgram,"specColor"),1,spCol); |
glUniform4f(glGetUniformLocation(shaderProgram,"specColor"),spCol[0],spCol[1],spCol[2],
spCol[3]);
glUniform1fv(glGetUniformLocation(shaderProgram,"t"),2,threshold);
glUniform4fv(glGetUniformLocation(shaderProgram,"color"),3,colors);
```

4.3. Sentències i funcions

Sentències de control de fluxe: Semblants a C:

```
if (bool expression)
  ...
else
...

for (initialization; bool expression; loop expression)
...

while (bool expression)
...

do
...
while (bool expression)
```

Instruccions de bifurcació:

- `continue;` Només en bucles. Bifurca a la següent iteració.
- `break;` Només en bucles. Sortida del bucle.
- `discard;` Només en fragment shaders. Finalitza execució del shader pel fragment que processa sense gravar al frame buffer.

4.3. Sentències i **funcions**

Com en C un shader s'estructura en funcions. Tot shader ha de tenir una funció principal declarada com:

```
void main()
{ <codi>
}
```

També es poden definir funcions en el shader. Com en C, una funció pot retornar un valor i s'utilitza la comanda

```
return <valor>;
```

per donar el resultat. Tota funció només pot retornar un tipus basic, **no un array.**

Una funció també pot ser `void`, sense retornar cap valor.

4.3. Sentències i **funcions**

Dues consideracions més:

- Una funció pot ser sobrecarregada de paràmetres
- No es poden definir funcions recursives

Exemple:

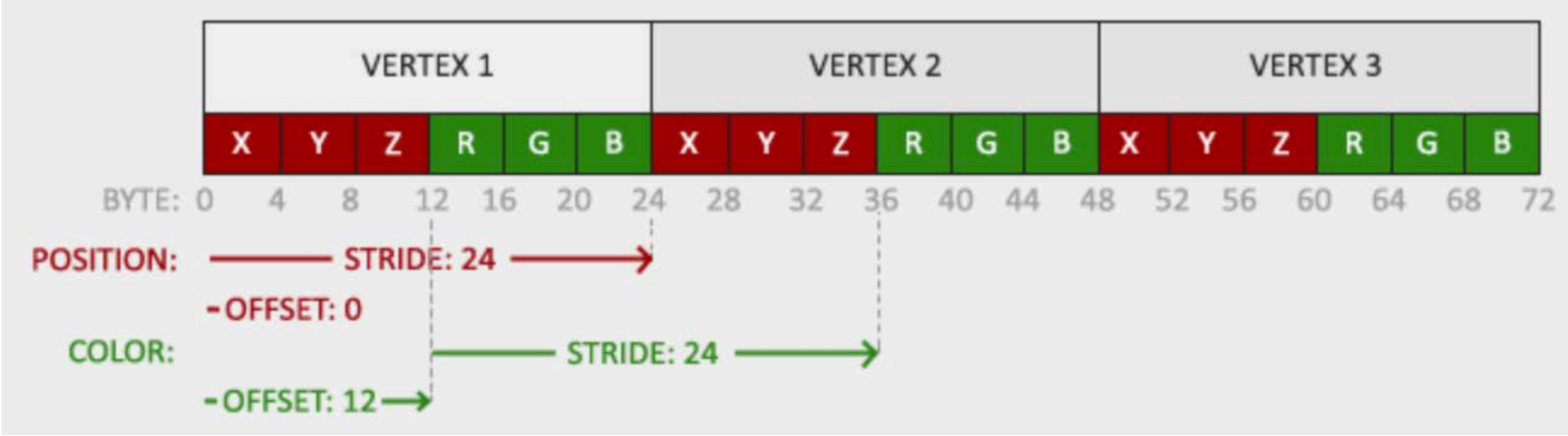
```
vec4 toonify(in float intensity)
{ vec4 color;
  if (intensity > 0.98) color = vec4(0.8,0.8,0.8,1.0);
   else if (intensity > 0.5) color = vec4(0.4,0.4,0.8,1.0);
         else if (intensity > 0.25) color = vec4(0.2,0.2,0.4,1.0);
               else   color = vec4(0.1,0.1,0.1,1.0);
  return color;
}
```

4.4. Exemple VS - FS

Exemple de triangle definit per vèrtexs i tres colors diferents.

Atributs del triangle:

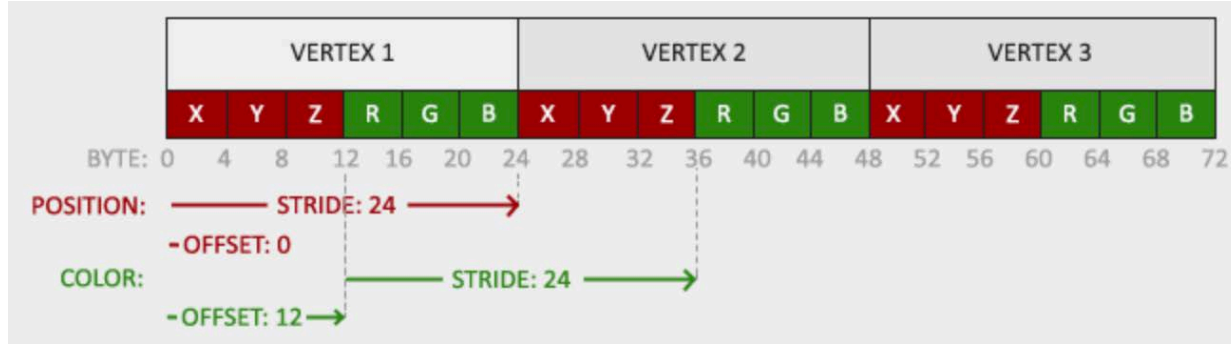
```
// set up vertex data (and buffer(s)) and configure vertex attributes
float vertices[] = { // Positions    // Colors
                      0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f, // Bottom right
                     -0.5f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f, // Bottom left
                      0.0f,  0.5f, 0.0f, 0.0f, 0.0f, 1.0f  // Top };
```



4.4. Exemple VS - FS

Exemple de triangle definit per vèrtexs i tres colors diferents.

Codi de l'aplicació:




```
unsigned int VBO, VAO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO); // bind the Vertex Array Object first, then bind and set vertex
                       //     buffer(s), and then configure vertex attributes(s).
glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
// Position attribute
glVertexAttribPointer(0,   3,   GL_FLOAT,   GL_FALSE,   6   *   sizeof(float),   (void*)0);
glEnableVertexAttribArray(0);
// Color attribute
glVertexAttribPointer(1,   3,   GL_FLOAT,   GL_FALSE,   6   *   sizeof(float),   (void*)(3   *
sizeof(float)));
glEnableVertexAttribArray(1);
```

4.4. Exemple VS - FS

Exemple de triangle definit per vèrtexs i tres colors diferents (2).

Vertex Shader:

```
#version 330 core
layout (location = 0) in vec3 aPos;      // vertices amb posició 0
layout (location = 1) in vec3 aColor;    // vertices amb posició 1
out vec3 ourColor; // Color de sortida al fragment shader
void main()
{ gl_Position = vec4(aPos, 1.0);
  ourColor = aColor; //Color del vèrtex és el color de l'objecte (aColor)
}
```

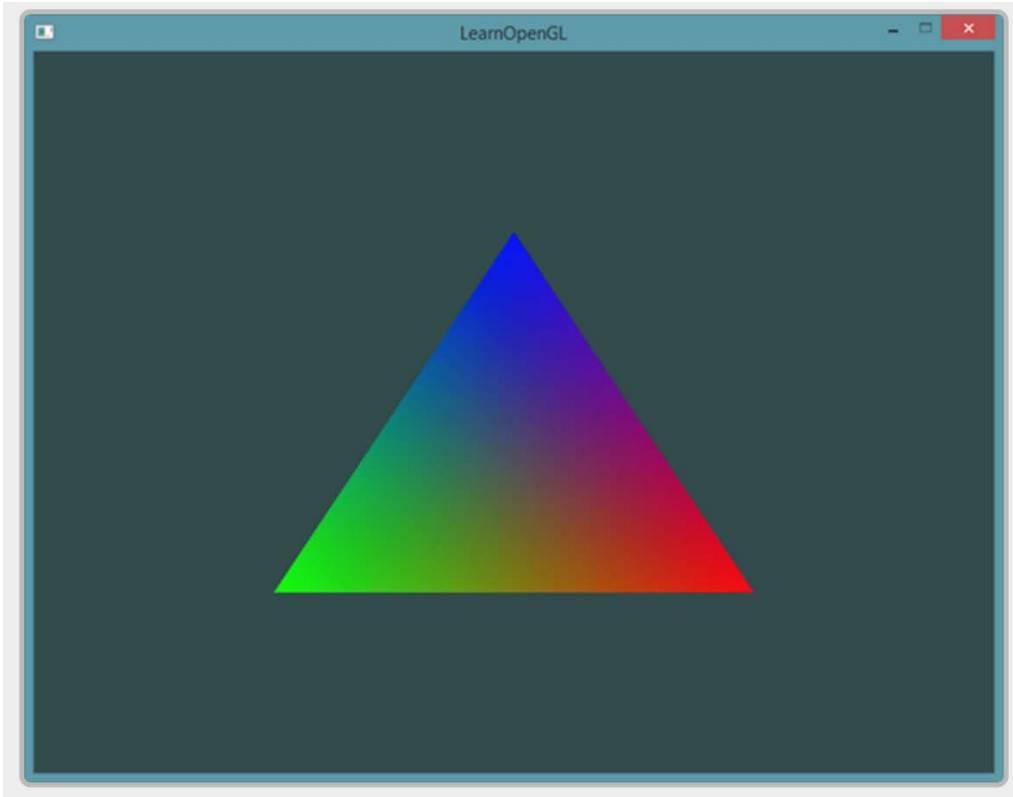
Fragment Shader:

```
#version 330 core
in vec3 ourColor;
out vec4 FragColor;
void main()
{   FragColor = vec4(ourColor, 1.0);
}
```

4.4. Exemple VS - FS

Exemple de triangle definit per vèrtexs i tres colors diferents (3).

Imatge:



Color diferent dels tres vèrtexs fa que cada fragment tingui un color interpolat a partir dels tres.

4.5. Compilació de Shaders

Compilació i linkedició en temps d'execució.

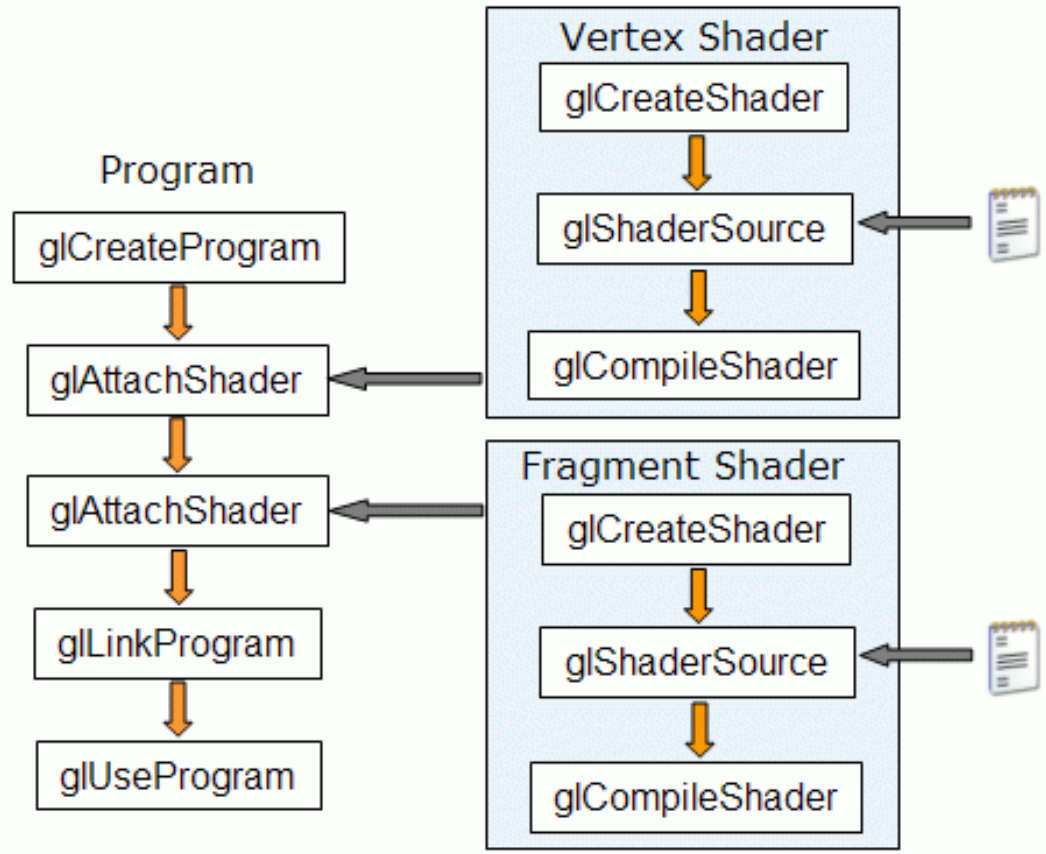
El codi font dels shaders usualment és en fitxers amb sufixes `.vert` i `.frag`

Usualment s'encapsula el procés de compilació i linkedició en una funció o clase.

Un cop compilat i linkat el shader, se li passen els valors a les variables `in` i `uniform` des de l'aplicació.

4.5. Compilació de Shaders

Passos per a crear i compilar shaders així com passar paràmetres als shaders:



4.5. Compilació de Shaders

Passos per a crear i compilar shaders, asociar-los a un programa i passar valors a les variables:

1. Llegir fitxers .vert, .frag → `textRead()`
2. Crear id's per a VS i FS → `glCreateShader()`
3. Compilar VS, FS → `glCompileShader()`
4. Crear id del program → `glCreateProgram()`
5. Assignar VS i FS al program → `glAttachShader()`
6. Linkar el program → `glLinkProgram()`
7. Esborrar id's de VS i FS → `glDeleteShader()`
8. Activar el program → `glUseProgram()`
9. Passar valor a les variables → `glUniform*()`

4.5. Compilació de Shaders

```
char* textFileRead(char *file);
```

Acció: Llegir fitxer de texte i retorna una cadena de caràcters amb el contingut del fitxer. Funció pròpia no estàndard.

Paràmetres:

`file`: Nom del fitxer que conté el codi font del shader.

4.5. Compilació de Shaders

```
char* textFileRead(char *file);
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
char *textFileRead(char *fn) {
FILE *fp;
char *content = NULL;
int count=0;
if (fn != NULL)
 { fp = fopen(fn,"rt");
   if (fp != NULL)
         {        fseek(fp, 0, SEEK_END);
                  count = ftell(fp);
                  rewind(fp);
                  if (count > 0) {   content = (char *)malloc(sizeof(char) * (count+1));
                                     count = fread(content,sizeof(char),count,fp);
                                              content[count] = '\0';
                                  }
                  fclose(fp);
         }
         else { fprintf(err, "Vertex or Fragment Program not found")); return NULL; }
 }
return content;
}
```

4.5. Compilació de Shaders

```
GLint glCreateShader(GLenum shaderType);
```

Acció: Crea un shader, segons el tipus (`shaderType`) i retorna un `GLint` amb l'identificador del shader creat.

Paràmetres:

`shaderType`: Identificador del shader que es crea. Pot tenir els valors:

- `GL_COMPUTE_SHADER`
- `GL_VERTEX_SHADER`
- `GL_TESS_CONTROL_SHADER`
- `GL_TESS_EVALUATION_SHADER`
- `GL_GEOMETRY_SHADER`
- `GL_FRAGMENT_SHADER`

4.5. Compilació de Shaders

```
void glShaderSource(GLuint shaderId, GLsizei count, const GLchar
**string, const GLint *length);
```

Acció: Associa el codi font de `string` al shader identificat per `shader`.

Paràmetres:

`shaderId`: Identificador del shader program.

`count`: Nombre d'elements en `string`. Usualment 1.

`string`: Array d'apuntadors a strings que contenen el codi font.

`lenght`: Array de longituds de string. Usualment NULL, el que significa que els strings acaben en NULL.

4.5. Compilació de Shaders

```
void glCompileShader(GLuint shaderId);
```

Acció: Compila el codi font associat al shader `shaderId`. El resultat de la compilació es pot consultar amb la comanda:

```
stat = glGetShader(GL_COMPILE_STATUS);
```

Si es retorna `GL_TRUE`, la compilació és sense errors. Si retorna `GL_FALSE`, s'han produït errors de compilació que es poden consultar amb la funció `glGetShaderInfoLog();`

Paràmetres:

`shaderId`: Identificador del shader program a compilar.

4.5. Compilació de Shaders

```
GLint glCreateProgram();
```

Acció: Crea un objecte program que inclourà els shaders compilats. Retorna en `GLint` l'identificador al programa creat.

4.5. Compilació de Shaders

```
void glAttachShader(Gluint programId, Gluint shaderId);
```

Acció: Associa l'objecte shader `shaderId` a l'objecte tipus program `programId`. A un mateix program caldrà associar els diferents objectes shader (VS i FS mínim) que volem aplicar. Per altra cantó, un mateix objecte shader es pot associar a diferents programs.

Paràmetres:

`programId`: Identificador del program a associar el shader.

`shaderId`: Identificador del shader a associar.

4.5. Compilació de Shaders

```
void glLinkProgram(GLuint programId);
```

Acció: Linka els diferents objectes shaders associats a `programId` generant els executables per a ser utilitzats. El resultat de la linkedició es pot consultar amb la comanda:

```
stat = glGetProgram(programId,GL_LINK_STATUS Glint result);
```

Si `result` retorna `GL_TRUE`, la linkedició és sense errors i totes les variables uniform del program s'inicialitzen a 0. Si retorna `GL_FALSE`, s'han produït errors de linkedició que es poden consultar amb la funció `glGetShaderInfoLog();`

Paràmetres:

`programId`: Identificador del program a linkar.

4.5. Compilació de Shaders

```
void glUseProgram(GLint programId);
```

Acció: Instal·la (o activa) l'executable del `programId` en el pipeline de visualització per a aplicar els shaders associats al program.

`glUseProgram(0);` en el perfil `ARB_COMPATIBILITY` es retorna a GL1.1.

Paràmetres:

`programId`: Identificador del shader program que volem activar.

```
void glDeleteProgram(GLint programId);
```

Acció: Elimina l'objecte program identificat com a `programId`.

Paràmetres:

`programId`: Identificador del program que volem eliminar.

4.5. Compilació de Shaders

Exemple. Funció que carrega i compila shaders, crea el program, associa els shaders, els linka i activa el program.

```
/******************************************************************************
* loadFileShaders(): Loads custom shader by specifying filename (expects frag/vert pair)
******************************************************************************/
GLuint loadFileShaders(char* file_Vert, char* file_Frag)
{   GLuint v, f, p;
    char *vs = NULL, *fs = NULL;
    int info_log_length = 0, compile_result = 0, link_result=0;
    FILE *fd;

    v = glCreateShader(GL_VERTEX_SHADER);
    f = glCreateShader(GL_FRAGMENT_SHADER);

    vs = textFileRead(file_Vert);
    fs = textFileRead(file_Frag);

    if ((vs == NULL) || (fs == NULL)) return GL_FALSE;      // Si no s'han trobat els fitxers, retornar 0

    const char * vv = vs;
    const char * ff = fs;
    glShaderSource(v, 1, &vv, NULL);
    glShaderSource(f, 1, &ff, NULL);

    free(vs);
    free(fs);
…
```

4.5. Compilació de Shaders

Exemple (2). Funció que carrega, i compila shaders així com crea el program, associa els shaders, els linka i activa el program.

```
// loadFileShaders(): Loads custom shader by specifying filename (expects frag/vert pair)
…
// Compilar Vertex Shader
    glCompileShader(v);
    glGetShaderiv(v, GL_COMPILE_STATUS, &compile_result);
    if (compile_result == GL_FALSE) {  // Llista error de compilació
            GLint maxLength = 0;
            glGetShaderiv(v, GL_INFO_LOG_LENGTH, &maxLength); // The maxLength includes the NULL character

            std::vector<GLchar> errorLog(maxLength);
            glGetShaderInfoLog(v, maxLength, &maxLength, &errorLog[0]);
            fprintf(ferr,Error compilació Vertex Shader\n"));
            // Volcar missatges error a fitxer GLSL_Error.VERT
            if ((fd = fopen("GLSL_Error.VERT", "w")) == NULL)
                        {  fprintf(stderr, "%s \n", "GLSL_Error.VERT was not opened"));
                           return GL_FALSE;
                        }
            for (int i = 0; i <= maxLength; i = i++) fprintf(fd, "%c", errorLog[i]);
            fclose(fd);

            glDeleteShader(v);      // Don't leak the shader.
            return GL_FALSE;                    // Exit with failure.
            }
…
```

4.5. Compilació de Shaders

Exemple (3). Funció que carrega, i compila shaders així com crea el program, associa els shaders, els linka i activa el program.

```
// loadFileShaders(): Loads custom shader by specifying filename (expects frag/vert pair)
…
// Compilar Fragment Shader
    glCompileShader(f);
    glGetShaderiv(f, GL_COMPILE_STATUS, &compile_result);
    if (compile_result == GL_FALSE){  // Llista errors de compilació
            GLint maxLength = 0;
            glGetShaderiv(f, GL_INFO_LOG_LENGTH, &maxLength); // The maxLength includes the NULL character

            std::vector<GLchar> errorLog(maxLength);
            glGetShaderInfoLog(f, maxLength, &maxLength, &errorLog[0]);
             fprintf(stderr, "%s \n","Error compilació Fragment Shader"));
            // Volcar missatges error a fitxer GLSL_Error.FRAG
            if ((fd = fopen("GLSL_Error.FRAG", "w")) == NULL)
                        { fprintf(stderr, "%s \n","GLSL_Error.FRAG was not opened"));
                           return GL_FALSE;
                        }
            for (int i = 0; i <= maxLength; i = i++) fprintf(fd, "%c", errorLog[i]);
            fclose(fd);

            glDeleteShader(f);      // Don't leak the shader.
            return GL_FALSE;        // Exit with failure.
            }
…
```

4.5. Compilació de Shaders

Exemple (4). Funció que carrega, i compila shaders creant el program, associa els shaders, els linka i activa el program.

```
// loadFileShaders(): Loads custom shader by specifying filename (expects frag/vert pair) (2)
…
// Creació del Program que inclou Vertex i Fragment Shader
    p = glCreateProgram();
    glAttachShader(p, v);            // Relaciona Vertex Shader al Program
    glAttachShader(p, f);            // relaciona Fragment Shader al Program
    glLinkProgram(p);                // Linkedició del Program
    glGetProgramiv(p, GL_LINK_STATUS, &link_result);
    if (link_result == GL_FALSE){    // Llista errors de linkedició del Program
            GLint maxLength = 0;
            glGeProgramiv(p, GL_INFO_LOG_LENGTH, &maxLength);
            // The maxLength includes the NULL character
            std::vector<GLchar> errorLog(maxLength);
            glGetProgramInfoLog(p, maxLength, &maxLength, &errorLog[0]);
            fprintf(stderr, "%s \n", “Error Linkedicio del Shader Program"));
            // Volcar missatges error a fitxer GLSL_Error.LINK
            if ((fd = fopen("GLSL_Error.LINK", "w")) == NULL)
            {  fprintf(stderr, "%s \n", "GLSL_Error.LINK was not opened");
               return GL_FALSE;     }
            for (int i = 0; i <= maxLength; i = i++) fprintf(fd, "%c", errorLog[i]);
            fclose(fd);
            glDeleteProgram(p);                 // Don't leak the shader.
            return GL_FALSE;        }           // Exit with failure.
    return p; // Retorna referència al Program creat
}
```

4.6. Shader Debugging

Compilació i linkedició en temps d'execució.

Comanda `glGetError()` no té accés a execució de codi GLSL.

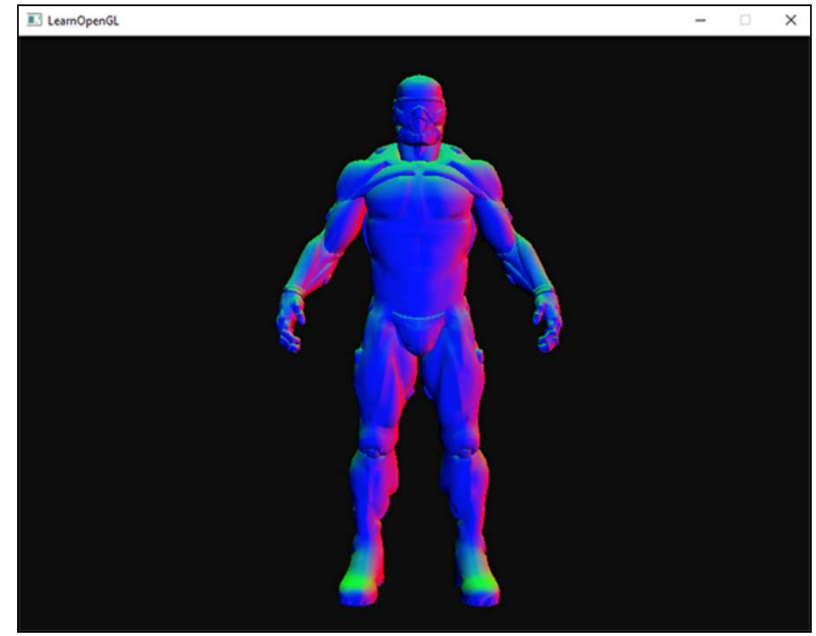
Podem detectar errors de compilació GLSL, però no errors semàntics, que donen usualment pantalla negre.

Truc: Si volem inspeccionar una variable del shader, podem enviar el valor de la variable al fragment i visualitzar el valor com a color en el frame buffer.

4.6. Shader Debugging

Truc: Visualitzar vector normal en frame buffer.

```
##version 330 core // Fragment shader
in vec3 Normal;
out vec4 FragColor;
void main()
{ FragColor.rgb = Normal;
  FragColor.a = 1.0;
}
```



Es veurà vector normal com a color en la imatge resultant. Si la imatge tota negra, segurament vectors normals no correctes (0,0,0).

4.6. Shader Debugging

Altres eines de debug:

1. **GLSL langvalidator:** Validador de codi GLSL
2. Eines de debug externes:
   a) RenderDoc
   b) CodeXL
   c) Nvidia Nsight

4.6.1. GLSL langvalidator

Cada driver té les seves característiques, segons el fabricant:

- Nvidia relaxa més les restriccions de l'especificació GLSL
- ATI/AMD són més restrictius en especificacions GLSL.

Per assegurar-se que el codi GLSL funciona en diferents implementacions: **GLSL langvalidator**

https://www.khronos.org/opengles/sdk/tools/Reference-Compiler/

4.6.1. GLSL langvalidator

GLSL langvalidator valida codi del shader en les especificacions GLSL que corresponguin segons la capçalera del codi i el sufix del fitxer:

- .vert - *vertex shader*
- .tesc - *tessellation control shader*
- .tese - *tessellation evaluation shader*
- .geom - *geometry shader*
- .frag - *fragment shader*
- .comp - *compute shader*

4.6.2. Eines de Debug Externes: RenderDoc

https://renderdoc.org/

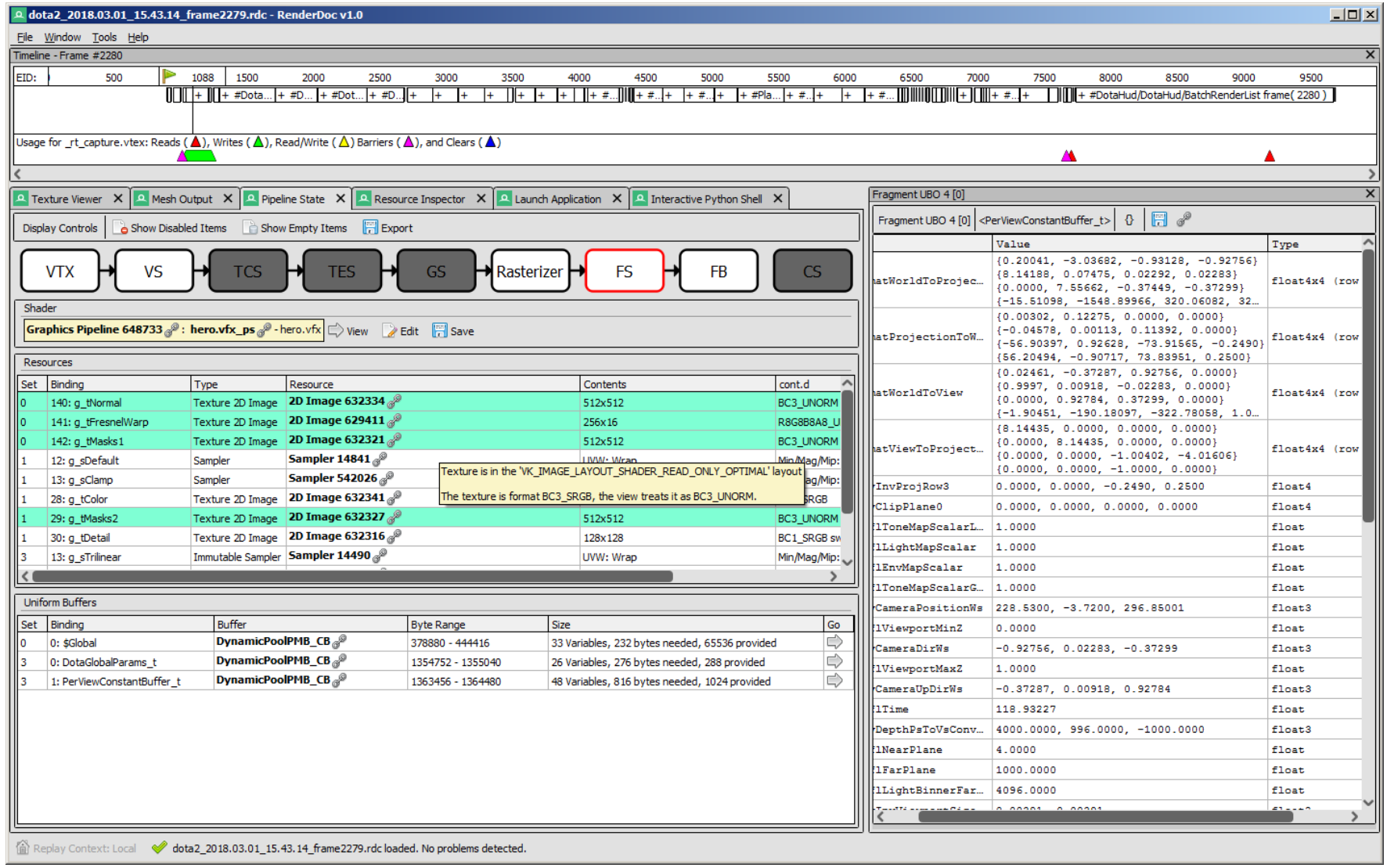
Eina de debugging opensource standalone, no plugin.

Cal especificar l'executable i directoris.

RenderDoc executa l'aplicació i captura un frame amb les seves variables.

Dins un frame es pot veure el pipeline, les comandes OpenGL, buffers i textures.

4.6.2. Eines de Debug Externes: RenderDoc



4.6.2. Eines de Debug Externes: CodeXL

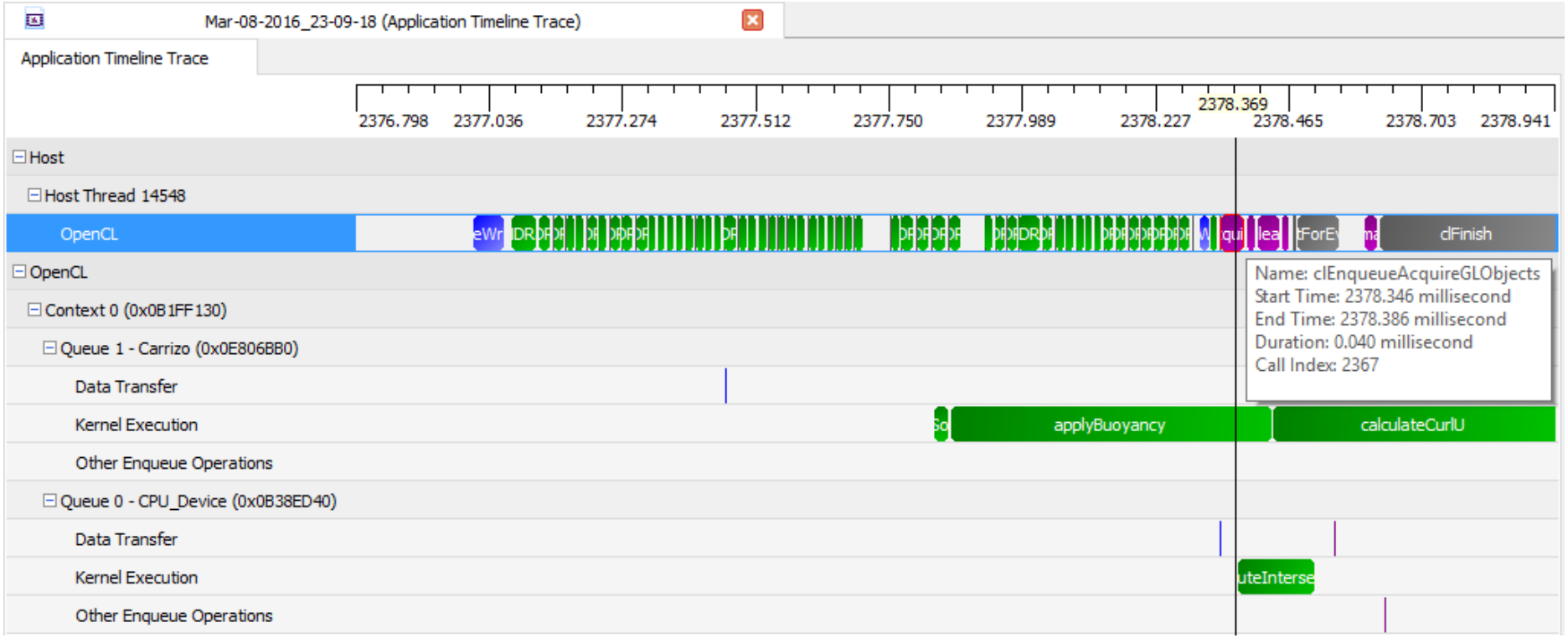
https://gpuopen.com/compute-product/codexl/

Eina de *debugging opensource standalone* (Windows i Linux), o *plugin* en diferents entorns com Visual Studio.

Proporciona informació útil i bo per a depurar shaders.

Treballa amb hardware gràfic de diferents fabricants (Nvidia, Intel, ATI, etc.).

4.6.2. Eines de Debug Externes: CodeXL



4.6.2. Eines de Debug Externes: Nsight

https://developer.nvidia.com/tools-overview

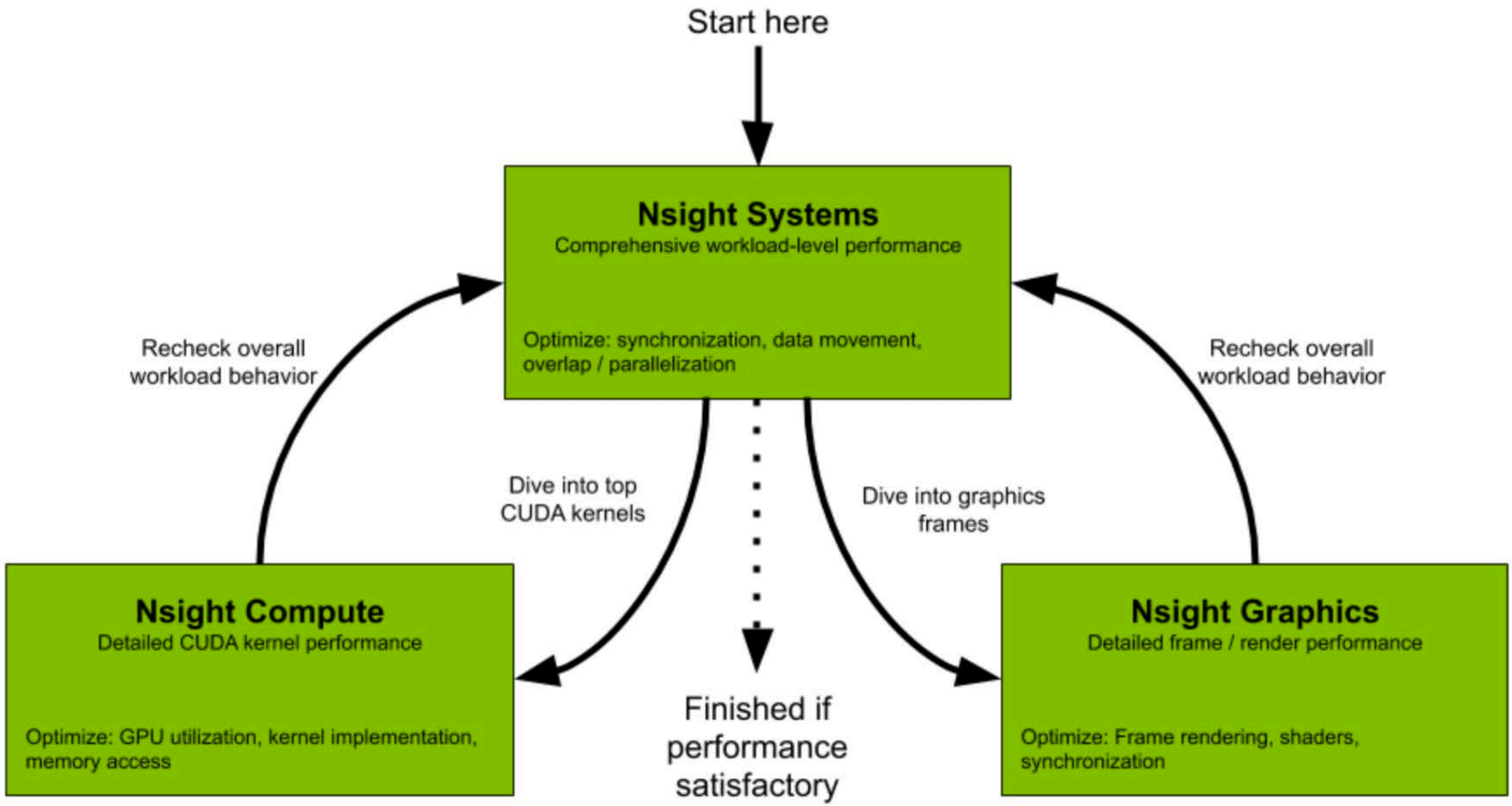
Plugin per a hardware gràfic Nvidia i entorns com Visual Studio o Eclipse.

Eina molt útil que dona molta informació, com ús de GPU's i frame a frame.

Quan s'executa l'aplicació en Visual Studio, Nsight s'executa dins el propi entorn

Molt bona eina de depuració, presenta una interessant GUI mostrant tot tipus d'informació *run-time* o per anàlisi *frame* a *frame*.

4.6.2. Eines de Debug Externes: Nsight



5. Exemples de Shaders

- Minimal Shader
- Flatten Shader
- Vertex Animation Shader

5.1. Minimal Shader

A minimal shader that performs the most basic tasks: transform the vertices and render the primitives in a single color.

VS:

```
#version 330 core
layout (location=0) in vec4 in_Vertex;
layout (location=1) in vec4 in_Color;

uniform mat4 modelMatrix;      // Matriu TG
uniform mat4 viewMatrix;       // Matriu Projecció
uniform mat4 projectionMatrix  // Matriu Projecció
out vec4 gl_Position;          // Coord. Pantalla (out-predefinida)
out vec4 color;                // Coord. Pantalla (out)

void main()
{  // Pas del vertex en Coord. Pantalla i color al FS
  gl_Position = projectionMatrix * viewMatrix * modelMatrix * in_Vertex;
  color = in_Color;
}
```

5.1. Minimal Shader

FS:

```
void main()
{
 gl_FragColor = vec4(0.4,0.3,0.8,1.0);
}
```

5.2. Flatten Shader

To flatten a 3D model, by setting its z coordinate to zero prior to applying the modelview transformation.



VS:

```
#version 330 core
layout (location=0) in vec4 in_Vertex;
layout (location=1) in vec4 in_Color;

uniform mat4 modelMatrix;        // Matriu TG
uniform mat4 viewMatrix;         // Matriu Projecció
uniform mat4 projectionMatrix    // Matriu Projecció

out vec4 gl_Position;            // Coord. Pantalla (out-predefinida)

void main()
{
  vec4 v = in_Vertex;
  v.z = 0.0;
  gl_Position = projectionMatrix * viewMatrix * modelMatrix * v;
}
```

5.2. Flatten Shader

FS:

```
#version 330 core

out vec4 gl_FragColor          // Color fragment (out)

void main()
{
  gl_FragColor = vec4(0.8,0.3,0.3,1.0);
}
```

5.3. Vertex Animation Shader

We need a variable to keep track of time, or a frame counter. We need to define this variable in the OpenGL application, and pass it to the shader as a uniform variable.

VS:

```
#version 330 core
layout (location=0) in vec4 in_Vertex;
layout (location=1) in vec4 in_Color;

uniform mat4 modelMatrix;        // Matriu TG
uniform mat4 viewMatrix;         // Matriu Projecció
uniform mat4 projectionMatrix    // Matriu Projecció
uniform float time;              // Temps del sistema

out vec4 gl_Position;            // Coord. Pantalla (out-predefinida)

void main()
{
  vec4 v = in_Vertex;
  v.z = v.z + (sin(time) + 1.0 ) / 2.0); // v.z = v.z + Tx
  gl_Position = projectionMatrix * viewMatrix * modelMatrix * v;
}
```

5.3. Vertex Animation Shader

FS:

```
#version 330 core
out vec4 gl_FragColor                          // Color fragment (out)

void main()
{
  gl_FragColor = vec4(0.8,0.3,0.3,1.0);
}
```

En aplicació:

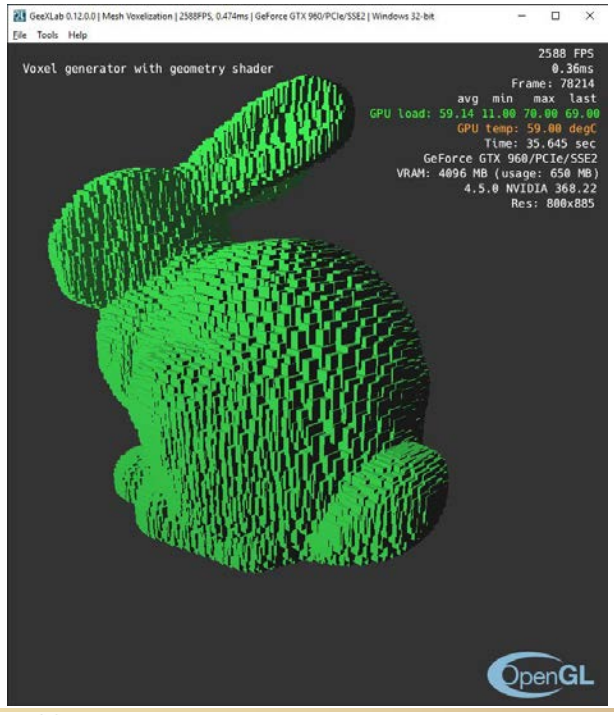
La variable `time` al VS es passa com uniform.

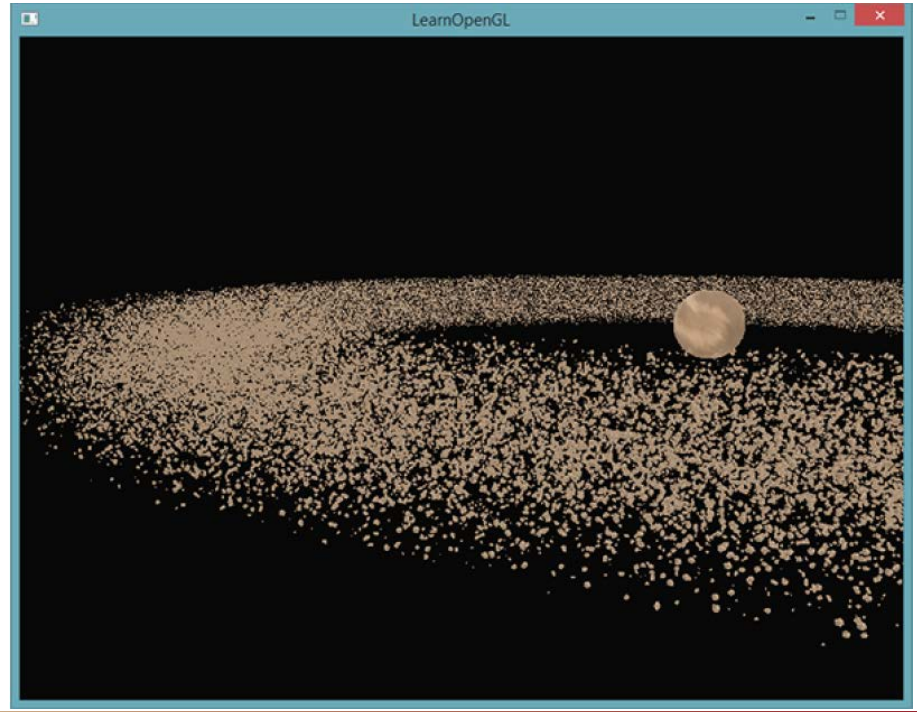
```
glUniform1f(glGetUniformLocation(shaderId,"time"), glfwGetTime()); // Llibreria GLFW
```

6. Geometry Shader (GS)

Shader entre VS i FS.

Pren un conjunt de vèrtexs que formen una primitiva (punts, línies, triangle) i modifica el tipus o forma de la primitiva, afegint o eliminant vèrtexs.





6. Geometry Shader (GS)

Compilació GS: Semblant a com es compila VS i FS. Cal lligar-lo a un program que ja contingui VS i FS.

```
/**************************************************************************
* loadFileAllShaders(): Loads custom shader by specifying filename (expects frag/geom/vert trio)
**************************************************************************/
GLuint loadFileAllShaders(char* file_Vert, char* file_Geom, char* file_Frag)
{   GLuint v, f, g, p;
    char *vs = NULL, *fs = NULL;, *gs = NULL;
    int info_log_length = 0, compile_result = 0, link_result=0;
    FILE *fd;

    v = glCreateShader(GL_VERTEX_SHADER);        f = glCreateShader(GL_FRAGMENT_SHADER);
    g = glCreateShader(GL_GEOMETRY_SHADER);

    vs = textFileRead(file_Vert);                fs = textFileRead(file_Frag);
    gs = textFileRead(file_Geom);

    if ((vs == NULL) || (fs == NULL)) return GL_FALSE;      // Si no s'han trobat els fitxers, retornar 0

    const char * vv = vs;                        const char * ff = fs;
    const char * gg = gs;
    glShaderSource(v, 1, &vv, NULL);             glShaderSource(f, 1, &ff, NULL);
    glShaderSource(g, 1, &gg, NULL);
    free(vs);                                    free(fs);
    free(gs);
[…]
```

6. Geometry Shader (GS)

Compilació GS (2)

```
// loadFileAllShaders(): Loads custom shader by specifying filename (expects frag/geom/vert trio) (2)
[…]
// Compilar Vertex Shader
    glCompileShader(v);
    glGetShaderiv(v, GL_COMPILE_STATUS, &compile_result);
    if (compile_result == GL_FALSE) {  // Llista error de compilació
            GLint maxLength = 0;
            glGetShaderiv(v, GL_INFO_LOG_LENGTH, &maxLength); // The maxLength includes the NULL character

            std::vector<GLchar> errorLog(maxLength);
            glGetShaderInfoLog(v, maxLength, &maxLength, &errorLog[0]);
            fprintf(ferr,Error compilació Vertex Shader\n"));
            // Volcar missatges error a fitxer GLSL_Error.VERT
            if ((fd = fopen("GLSL_Error.VERT", "w")) == NULL)
                        {  fprintf(stderr, "%s \n", "GLSL_Error.VERT was not opened"));
                           return GL_FALSE;
                        }
            for (int i = 0; i <= maxLength; i = i++) fprintf(fd, "%c", errorLog[i]);
            fclose(fd);

            glDeleteShader(v);                  // Don't leak the shader.
            return GL_FALSE;                    // Exit with failure.
            }
[…]
```

6. Geometry Shader (GS)

Compilació GS (3)

```
// loadFileAllShaders(): Loads custom shader by specifying filename (expects frag/geom/vert trio)
[…]
// Compilar Geom Shader
    glCompileShader(g);
    glGetShaderiv(g, GL_COMPILE_STATUS, &compile_result);
    if (compile_result == GL_FALSE) {  // Llista error de compilació
             GLint maxLength = 0;
             glGetShaderiv(g, GL_INFO_LOG_LENGTH, &maxLength); // The maxLength includes the NULL character

             std::vector<GLchar> errorLog(maxLength);
             glGetShaderInfoLog(g, maxLength, &maxLength, &errorLog[0]);
             fprintf(ferr,Error compilació Geometry Shader\n"));
             // Volcar missatges error a fitxer GLSL_Error.GEOM
             if ((fd = fopen("GLSL_Error.GEOM", "w")) == NULL)
                        {  fprintf(stderr, "%s \n", "GLSL_Error.GEOM was not opened"));
                           return GL_FALSE;
                        }
             for (int i = 0; i <= maxLength; i = i++) fprintf(fd, "%c", errorLog[i]);
             fclose(fd);

             glDeleteShader(g);                  // Don't leak the shader.
             return GL_FALSE;                    // Exit with failure.
             }
[…]
```

6. Geometry Shader (GS)

Compilació GS (4)

```
// loadFileAllShaders(): Loads custom shader by specifying filename (expects frag/geom/vert trio)
[…]
// Compilar Fragment Shader
    glCompileShader(f);
    glGetShaderiv(f, GL_COMPILE_STATUS, &compile_result);
    if (compile_result == GL_FALSE){  // Llista errors de compilació
             GLint maxLength = 0;
             glGetShaderiv(f, GL_INFO_LOG_LENGTH, &maxLength); // The maxLength includes the NULL character

             std::vector<GLchar> errorLog(maxLength);
             glGetShaderInfoLog(f, maxLength, &maxLength, &errorLog[0]);
              fprintf(stderr, "%s \n","Error compilació Fragment Shader"));
             // Volcar missatges error a fitxer GLSL_Error.FRAG
             if ((fd = fopen("GLSL_Error.FRAG", "w")) == NULL)
                         { fprintf(stderr, "%s \n","GLSL_Error.FRAG was not opened"));
                            return GL_FALSE;
                         }
             for (int i = 0; i <= maxLength; i = i++) fprintf(fd, "%c", errorLog[i]);
             fclose(fd);

             glDeleteShader(f);      // Don't leak the shader.
             return GL_FALSE;        // Exit with failure.
             }
…
```

6. Geometry Shader (GS)

Compilació GS (5)

```
// loadFileAllShaders(): Loads custom shader by specifying filename (expects frag/geom/vert trio)
…
// Creació del Program que inclou Vertex, Geometry i Fragment Shader
    p = glCreateProgram();
    glAttachShader(p, v);           // Relaciona Vertex Shader al Program
    glAttachShader(p, f);           // Relaciona Fragment Shader al Program
    glAttachShader(p, g);           // Relaciona Geometry Shader al Program
    glLinkProgram(p);               // Linkedició del Program
    glGetProgramiv(p, GL_LINK_STATUS, &link_result);
    if (link_result == GL_FALSE){   // Llista errors de linkedició del Program
             GLint maxLength = 0;
             glGeProgramiv(p, GL_INFO_LOG_LENGTH, &maxLength);
             // The maxLength includes the NULL character
             std::vector<GLchar> errorLog(maxLength);
             glGetProgramInfoLog(p, maxLength, &maxLength, &errorLog[0]);
             fprintf(stderr, "%s \n", “Error Linkedicio del Shader Program"));
             // Volcar missatges error a fitxer GLSL_Error.LINK
             if ((fd = fopen("GLSL_Error.LINK", "w")) == NULL)
             {  fprintf(stderr, "%s \n", "GLSL_Error.LINK was not opened");
                return GL_FALSE;     }
             for (int i = 0; i <= maxLength; i = i++) fprintf(fd, "%c", errorLog[i]);
             fclose(fd);
             glDeleteProgram(p);                // Don't leak the shader.
             return GL_FALSE;        }          // Exit with failure.
    return p; // Retorna referència al Program creat
}
```

6. Geometry Shader (GS)

Exemples:

1. Crear noves geometries
2. Explosionar objectes
3. Visualitzar vectors normals d'un objecte

6.1. Crear noves geometries

Exemple de GS:

```
0: #version 330 core
1: layout (points) in;
2: layout (line_strip, max_vertices = 2) out;
3:
4: void main()
5: {
6:   gl_Position = gl_in[0].gl_Position + vec4(-0.1, 0.0, 0.0, 0.0);
7:   EmitVertex();
8:   gl_Position = gl_in[0].gl_Position + vec4(0.1, 0.0, 0.0, 0.0);
9:   EmitVertex();
10:  EndPrimitive();
11:}
```

Comentaris del codi GLSL:

```
0: #version 330 core
```

Versió del llenguatge GLSL.

6.1. Crear noves geometries

```
1: layout (points) in;
```

Declarem el tipus de primitiva que arriba al shader amb la directiva “`in`”.

Comanda `layout(input_primitive) in;`

`input_primitive` pot tenir els següents valors, segons la directiva donada a l’hora de dibuixar els vèrtexs amb la comanda `glDrawArrays()`:

input_primitive	Primitiva glDrawArrays()	# vèrtexs mínim
points	GL_POINTS	1
lines	GL_LINES, GL_LINE_STRIP, GL_LINE_LIST	2
line_adjacency	GL_LINES_ADJACENCY, GL_LINE_STRIP_ADJACENCY	4
triangles	GL_TRIANGLES, GL_TRIANGLE_STRIP, GL_TRIANGLE_FAN	3
triangles_adjacency	GL_TRIANGLES_ADJACENCY, GL_TRIANGLE_STRIP_ADJACENCY	6

6.1. Crear noves geometries

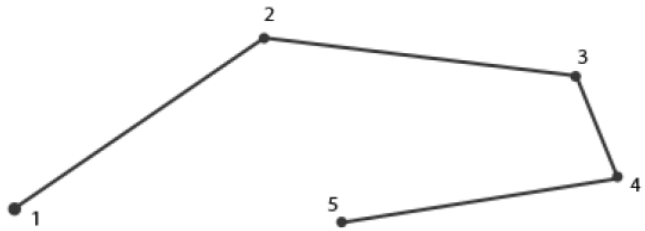
```
2: layout (line_strip, max_vertices = 2) out;
```

Tipus de primitiva de sortida amb la directiva “out”.

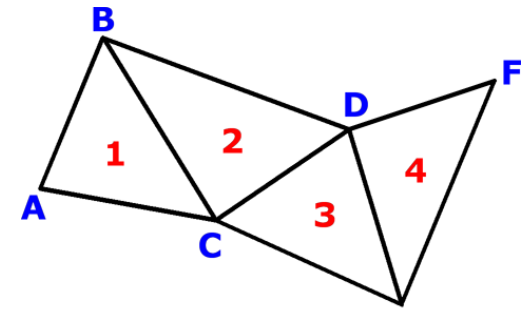
Comanda `layout(output_primitive, max_vertices = vert_count) out;`

Paràmetres:

- `output_primitive`: Tipus de primitiva de sortida. Pot tenir els valors:
  - `points:` Seqüència de punts
  - `line_strip:` Seqüència de línies enganxada.
  - `triangle_strip:` Seqüència de triangles enganxat.
- `max_vertices`: Màxim nombre de vèrtexs de la primitiva de sortida, definit en `vert_count.` Si en el codi ens excedim d’aquest nombre de vèrtexs els shader no els dibuixa.



`line_strip`



`triangle_strip`

En la comanda s’especifica que GS treurà una línia de dos vèrtexs.

6.1. Crear noves geometries

```
6:   gl_Position = gl_in[0].gl_Position + vec4(-0.1, 0.0, 0.0, 0.0);
8:   gl_Position = gl_in[0].gl_Position + vec4(0.1, 0.0, 0.0, 0.0);
```

GS té una variable predefinida d'entrada (*built-in*) que conté informació dels vèrtexs d'entrada, que té el format:

```
in gl_PerVertex
{ vec4 gl_Position;         // Coordenades del vèrtex.
  float gl_PointSize;       // Mida del vèrtex en píxels.
  float gl_ClipDistance[];  // Distància del vèrtex respecte cada pla del V. Visual: (0, +, -)
}  gl_in[];
```

`gl_in[]` és un vector doncs conté un nombre variable de vèrtexs corresponents a la primitiva d'entrada.

I una variable predefinida de sortida (*built-out*) que conté la coordenada del vèrtex de sortida.

```
out gl_PerVertex
{ vec4 gl_Position;           // Coordenades del vèrtex.
  float gl_PointSize;         // Mida del vèrtex en píxels.
  float gl_ClipDistance[]; }; // Distància del vèrtex respecte cada pla del V. Visual: (0, +, -)
};
```

Codi GS genera una línia sobre coordenada X de longitud 0.2 centrada en el punt d'entrada al shader.

6.1. Crear noves geometries

```
7:   EmitVertex();
9:   EmitVertex();
```

Després de definir un vèrtex a la variable `gl_Position` aquesta comanda l'afegeix a la primitiva de sortida

Nombre de vèrtexs de la primitiva de sortida ve definit pel nombre de crides a aquesta comanda.

```
11:  EndPrimitive();
```

Un cop definits els vèrtexs de sortida, aquesta els agrupa en un bloc i els passa al FS.

6.1. Crear noves geometries

Exemple simple:

Amb aquest GS, donat el següent codi en l'aplicació:

```
GLfloat points[] = {
-0.5f,  0.5f,      // Top-left
 0.5f,  0.5f,      // Top-right
 0.5f, -0.5f,      // Bottom-right
-0.5f, -0.5f       // Bottom-left
};
<Generar VAO i VBO sobre points>
<Carregar VS, GS, FS>
glUseProgram(shaderId);
glBindvertex(VAOId);
glDrawArrays(GL_POINTS,0,4);
glBindVertexArray(0);
```

6.1. Crear noves geometries

Exemple simple(2)

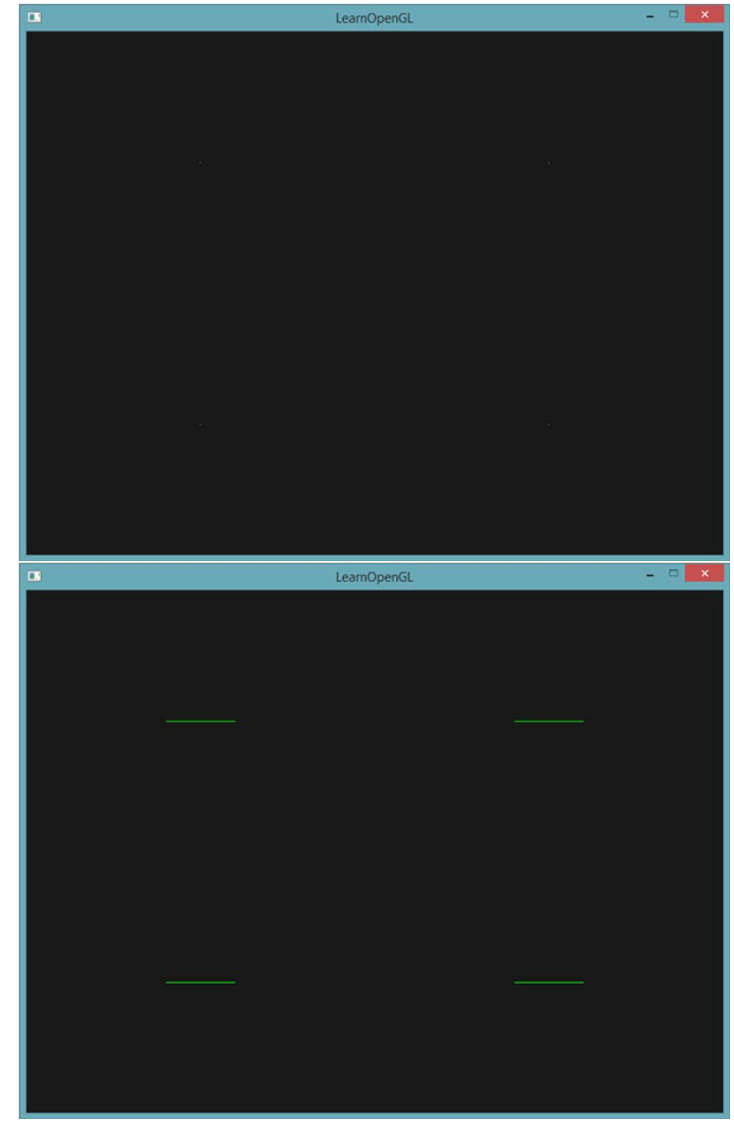
I el següent VS i FS:

VS:

```
#version 330 core
layout (location=0) in vec2 in_Vertex;
void main()
{  gl_Position = vec4(in_Vertex.x, in_Vertex.y,0.0,1.0);
}
```

FS:

```
#version 330 core
out vec4 FragColor;
void main()
{  FragColor = vec4(0.0,1.0,0.0,1.0); // Color verd
}
```



6.1. Crear noves geometries

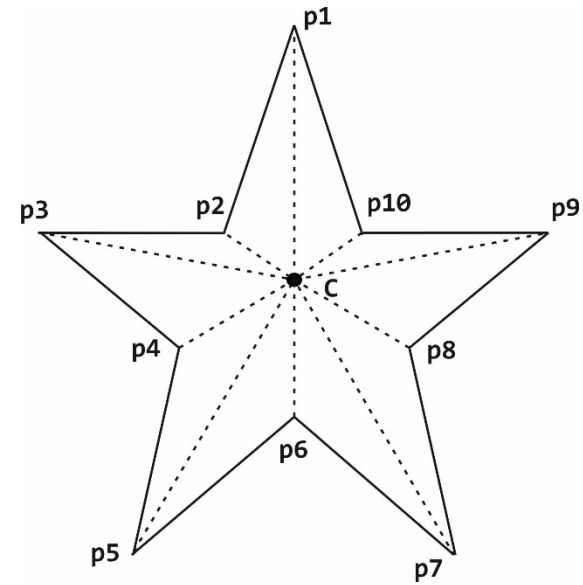
GS per defecte (simple):

```
#version 330 core
layout (points) in;
layout (points, max_vertices=1) out;
void main()
{
  glPosition = gl_in[0].glPosition;
  EmitVertex();
  EmitPrimitive();
}
```

6.1. Crear noves geometries

Exemple Estrella:

Per a cada punt, definir una geometria més complexe: Una estrella de 5 puntes dibuixada amb triangles formats per dos punts de l'estrella i el punt central (C), d'entrada al GS.




```
PUNTS:
p1=C+(0.0,5.5); p2=C+(-1.5,1.5); p3=C+(-5.5,1.5); p4=C+(-2.5,-1.0); p5=C+(-3.5,-5.5);
p6=C+(0.0,-2.0);p7=C+(3.5,-5.5); p8=C+(2.5,-1.0); p9=C+(5.5,1.5); p10=C+(1.5,1.5)
TRIANGLES:
p1-p2-C; p2-p3-C; p3-p4-C; Pp-p5-C; p5-p6-C;
p6-p7-C; p7-p8-C; p8-p9-C; Pp-p10-C; p10-p1-C;
```

6.1. Crear noves geometries

Exemple Estrella (2):

En aplicació:

```
GLfloat points[] = {
-0.5f,  0.5f,      // Top-left
 0.5f,  0.5f,      // Top-right
 0.5f, -0.5f,      // Bottom-right
-0.5f, -0.5f       // Bottom-left
};
<Generar VAO i VBO sobre points>
<Carregar VS, GS, FS>
glUseProgram(shaderId);
glBindVertex(VAOId);
glDrawArrays(GL_POINTS,0,4);
glBindVertexArray(0);
```

6.1. Crear noves geometries

Exemple Estrella (3):

GS:

```
#version 330 core
layout (points) in;
layout (triangle_strip,max_vertices=30) out;

void build_Star(vec4 position)
{ gl_Position = position + vec4(0.0,0.5,0.0,0.0); // p1
  EmitVertex();
  gl_Position = position + vec4(1.0,1.5,0.0,0.0); // p2
  EmitVertex();
  gl_Position = position;                        // C
  EmitVertex();
  [...]
  gl_Position = position + vec4(1.5,1.5,0.0,0.0); // p10
  EmitVertex();
  gl_Position = position + vec4(0.0,0.5,0.0,0.0); // p1
  EmitVertex();
  gl_Position = position + vec4(5.0,1.5,0.0,0.0); // C
  EmitVertex();
  EndPrimitive();
}

void main()
{ buid_Star(gl_in[0].gl_Position);
}
```

6.1. Crear noves geometries

Exemple Estrella Color: Introduir color en GS.

En aplicació:

```
GLfloat points[] = {
-0.5f,  0.5f, 1.0f, 0.0f, 0.0f,      // Top-left, color vermell
 0.5f,  0.5f, 0.0f, 1.0f, 0.0f,      // Top-right, color verd
 0.5f, -0.5f, 0.0f, 0.0f, 1.0f,      // Bottom-right, color blau
-0.5f, -0.5f, 1.0f, 1.0f, 1.0f      // Bottom-left, color blanc
};
<Generar VAO i VBO sobre points>
<Carregar VS, GS, FS>
glUseProgram(shaderId);
glBindVertex(VAOId);
glDrawArrays(GL_POINTS,0,4);
glBindVertexArray(0);
```

6.1. Crear noves geometries

Exemple Estrella Color (2):

VS:

```
#version 330 core
layout (location=0) in vec4 in_Vertex;
layout (location=1) in vec4 in_Color;

out VS_OUT{
  vec3 color;
} vs_out;

void main()
{ gl_Position = vec4(in_Vertex.x, in_Vertex.y, 0.0, 1.0);
  vs_out.color = in_Color;
}
```

6.1. Crear noves geometries

Exemple Estrella Color (3):

GS:

```
#version 330 core
layout (points) in;
layout (triangle_strip,max_vertices=30) out;
in VS_OUT{
  vec3 color;
} gs_in[];
out vec3 fColor;

void main()
{
  fColor = gs_in[0].color;                          // Color del punt
  gl_Position = gl_in[0].gl_Position + vec4(0.0,5.5,0.0,0.0);  // p1
  EmitVertex();
  gl_Position = gl_in[0].gl_Position + vec4(-1.5,1.5,0.0,0.0); // p2
  EmitVertex();
  gl_Position = gl_in[0].gl_Position;                          // C
  EmitVertex();
  [...]
  gl_Position = gl_in[0].gl_Position + vec4(1.5,1.5,0.0,0.0);  // p10
  EmitVertex();
  gl_Position = gl_in[0].gl_Position + vec4(0.0,5.5,0.0,0.0);  // p1
  EmitVertex();
  gl_Position = gl_in[0].gl_Position;                          // C
  EmitVertex();
  EndPrimitive();
}
```

6.1. Crear noves geometries

Exemple Estrella Color Diferent: Introduir color diferent a cada nou vèrtexs en el GS. Punt P1 a blanc.

GS:

```
#version 330 core
layout (points) in;
layout (triangle_strip,max_vertices=30) out;
in VS_OUT{
  vec3 color;
} gs_in[];
out vec3 fColor;

void main()
{ fColor = vec4 (1.0,1.0,1.0,1.0);                              // Color blanc
  gl_Position = gl_in[0].gl_Position + vec4(0.0,5.5,0.0,0.0); // P1
  EmitVertex();
  fColor = gs_in[0].color;                                      // Color del punt
  gl_Position = gl_in[0].gl_Position + vec4(-1.5,1.5,0.0,0.0);          // P2
  EmitVertex();
  gl_Position = gl_in[0].gl_Position;                                   // point
  EmitVertex();
  [...]
  gl_Position = gl_in[0].gl_Position + vec4(1.5,-1.5,0.0,0.0);          // P10
  EmitVertex();
  fColor = vec4 (1.0,1.0,1.0,1.0);                              // Color blanc
  gl_Position = gl_in[0].gl_Position + vec4(0.0,5.5,0.0,0.0); // P1
  EmitVertex();
  gl_Position = gl_in[0].gl_Position                             // point
  EmitVertex();
  EndPrimitive();        }
```

6.2. Explosionar objectes

Aplicat sobre la geometria d'un objecte, equival a desplaçar cada triangle en direcció el seu vector normal.

Donat un triangle (`gl_in[0]`, `gl_in[1]`, `gl_in[2]`) introduït al GS cal fer en aquest shader els següents càlculs:

1. Calcular vector normal del triangle
2. Calcular la direcció de desplaçament

6.2.1. Calcular el vector normal del triangle

GS:

Funció `GetNormal()`, que calcula el vector normal entre els tres vèrtexs a partir dos vectors a,b

```
vec3 GetNormal()
{
 vec3 a = vec3 (gl_in[0].gl_Position) – vec3 (gl_in[1].gl_Position);
 vec3 b = vec3 (gl_in[2].gl_Position) – vec3 (gl_in[1].gl_Position);
 return normalize(cross(a,b));
}
```

6.2.2. Calcular direcció de desplaçament

GS:

Funció `explode()`, que calcula la direcció i magnitud del desplaçament del vèrtex.

```
vec3 explode(vec4 position, vec3 normal)
{
 float magnitude = 2.0;
 vec3 direction = normal * (sin(time) + 1.0 ) / 2.0) * magnitude;
 return position + vec4(direction,1.0);
}
```

Per a donar-li component aleatòria, utilitzem la variable `time` que ens retorna el temps del sistema.

A time li apliquem la funció `sin()` que ens retorna un valor entre [-1,1] que convertim a [0,1] per a que cap triangle “implosioni”.

6.2. Explosionar objectes

GS:

```
#version 330 core
layout (triangles) in;
layout (triangle_strip,max_vertices=3) out;

in VS_OUT{
  vec2 texCoords;
} gs_in[];

out vec2 TexCoords;

uniform float time;

vec3 GetNormal()
{ vec3 a = vec3 (gl_in[0].glPosition) – vec3 (gl_in[1].glPosition);
  vec3 b = vec3 (gl_in[2].glPosition) – vec3 (gl_in[1].glPosition);
  return normalize(cross(a,b));
}
vec3 explode(vec4 position, vec3 normal)
{ float magnitude = 2.0;
  vec3 direction = normal * (sin(time) + 1.0 ) / 2.0) * magnitude;
  return position + vec4(direction,1.0);
}
/..
```

6.2. Explosionar objectes

GS (2):

```
/..
void main()
{
  vec3 normal = GetNormal();                              // Vector Normal del triangle.

  // Càlcul vèrtex gl_in[0]
  vec3 direction = explode(gl_in[0].glPosition, normal);          // Direcció desplaçament.
  glPosition = gl_in[0].glPosition + vec4(direction,1.0);         // desplaçament vèrtex.
  TexCoords = gs_in[0].texCoords;                                 // Coordenades textura del vèrtex.
  EmitVertex();

  // Càlcul vèrtex gl_in[1]
  vec3 direction = explode(gl_in[1].glPosition, normal);          // Direcció desplaçament.
  glPosition = gl_in[1].glPosition + vec4(direction,1.0);         // desplaçament vèrtex.
  TexCoords = gs_in[1].texCoords;                                 // Coordenades textura del vèrtex.
  EmitVertex();

  // Càlcul vèrtex gl_in[2]
  vec3 direction = explode(gl_in[2].glPosition, normal);          // Direcció desplaçament.
  glPosition = gl_in[2].glPosition + vec4(direction,1.0);         // desplaçament vèrtex.
  TexCoords = gs_in[2].texCoords;                                 // Coordenades textura del vèrtex.
  EmitVertex();

  EndPrimitive();
}
```

6.2. Explosionar objectes

En aplicació:

La variable `time` al GS es passa com uniform.

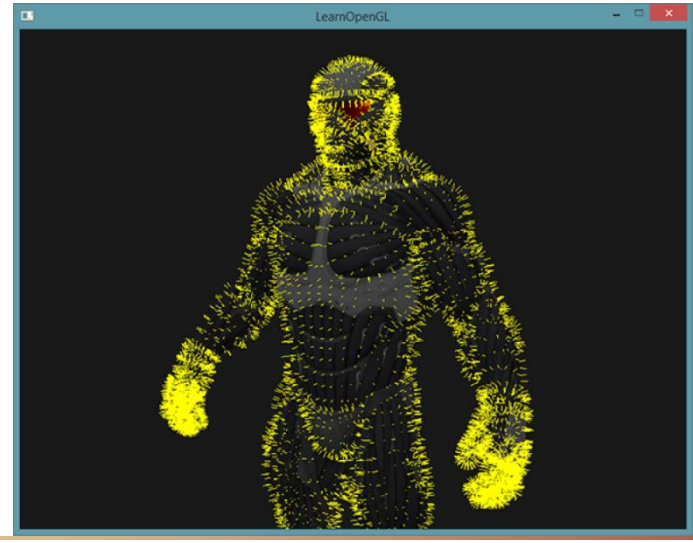
```
glUniform1f(glGetUniformLocation(shaderId,”time”), glfwGetTime());
```

6.3. Visualitzar vectors normal d'un objecte

Visualitzar el vector normal de cada vèrtex d'un objecte

Passos:

1. Dibuixar tots els triangles de l'objecte (amb VS, FS i sense GS)
2. Dibuixar vector normal per a cada vèrtex (amb VS, GS i FS propis)



6.3.2 Dibuixar vectors normal d'un vèrtex

Definir VS, GS i FS propis per s dibuixar vector normal de cada vèrtex.

En aplicació:

```
Shader.use();           // Activar shader dibuix objectes (VS i FS)
DrawScene();            // Dibuix dels objectes
normalDisplayShader.Use(); // Activar shader dibuix Vnormals (VS, GS, FS)
DrawScene();             // Dibuix dels objectes
```

6.3.2 Dibuixar vectors normal d'un vèrtex

Definir VS, GS i FS propis per a dibuixar vector normal de cada vèrtex.

VS:

```
#version 330 core
layout (location=0) in vec4 in_Vertex;
layout (location=1) in vec4 in_Color;

out VS_OUT{
  vec3 color;
} vs_out;

uniform mat4 projectionMatrix;
uniform mat4 viewMatrix;
uniform mat4 modelMatrix;
uniform mat4 normalMatrix;

void main()
{
  gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(in_Vertex,1.0);
  vs_out.normal = normalize(vec3(projectionMatrix * vec4(normalMatrix * normal), 1.0);
}
```

6.3.2 Dibuixar vectors normal d'un vèrtex

GS:

```
#version 330 core
layout (triangles) in;
layout (triangle_strip,max_vertices=3) out;

in VS_OUT{
  vec2normal;
} gs_in[];

const float MAGNITUDE = 0.4f;

void GenerateLine(int index)
{
  gl_Position = gl_in[index].gl_Position;
  EmitVertex();
  gl_Position = gl_in[index].gl_Position + vec4(gs_in[index].normal, 0.0f) * MAGNITUDE;
  EmitVertex();
  EndPrimitive();
}

void main()
{
  GenerateLine(0); // First vertex normal
  GenerateLine(1); // Second vertex normal
  GenerateLine(2); // Third vertex normal
}
```

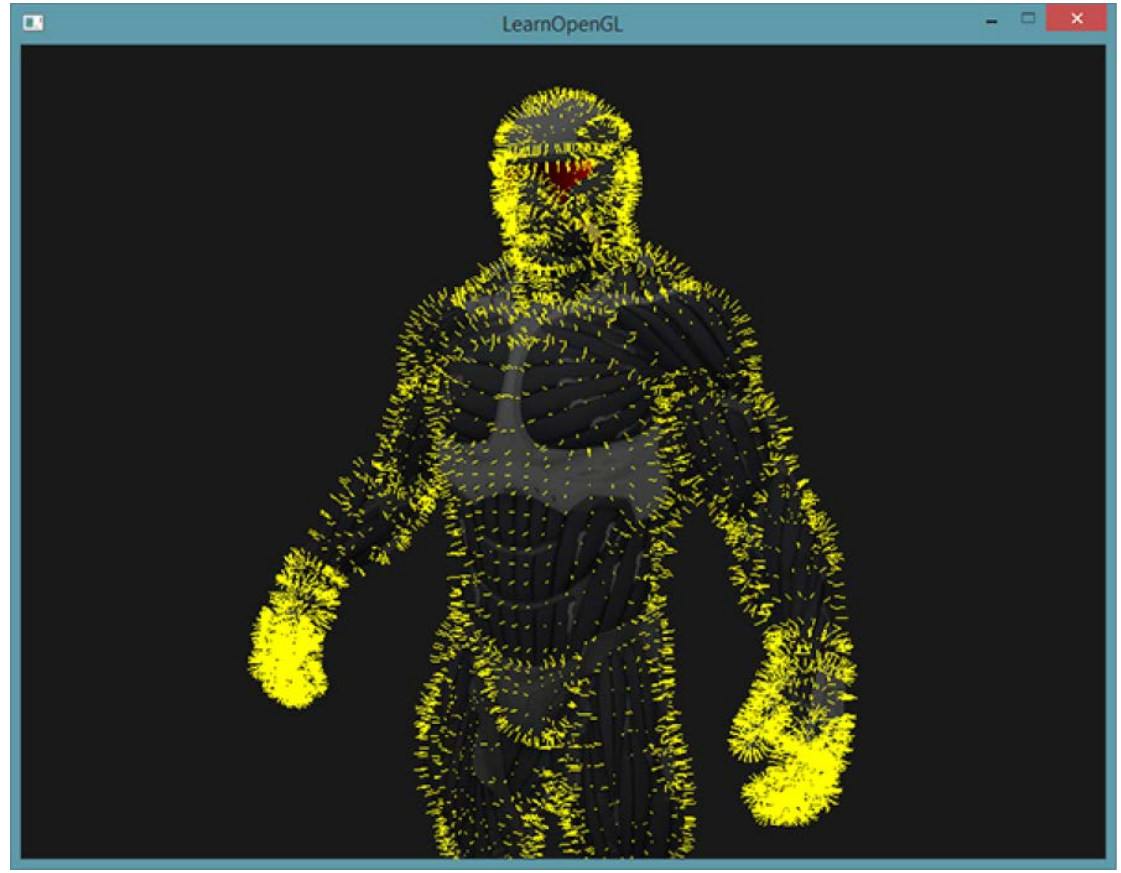
6.3.2 Dibuixar vectors normal d'un vèrtex

FS:

```
#version 330 core
out vec4 FragColor;

void main()
{
  FragColor = vec4 (1.0, 1.0, 0.0, 1.0);

}
```



En resum...

- Shaders són programes aplicats a cada component bàsica de visualització (vèrtexs, primitiva, píxel) que es poden executar de forma paralela en la placa gràfica.
- Shaders trenquen el rígid pipeline de visualització, permetent programar les *Graphical Processing Units (GPU)* de la placa gràfica, descarregant la CPU de la programació gràfica.
- Shaders són compilats cada cop el que permet aprofitar al màxim les característiques del hardware gràfic.
- Nous dispositius (mòbils, tauletes, etc.) ja tenen l'arquitectura gràfica programable amb shaders, quedant cada cop el rígid pipeline en desús.